final-melhorado-v2

June 15, 2026

[1]:
```
'''
Bibliotecas usadas
'''

import os
from datetime import datetime
import re
import subprocess
import warnings
import numpy as np
import pandas as pd
from collections import Counter
from Bio import Entrez
from Bio import SeqIO
from Bio.Seq import Seq
from sklearn.model_selection import (
    GroupShuffleSplit,
    StratifiedGroupKFold,
    RandomizedSearchCV,
    GroupKFold
    )
from sklearn.metrics import (
    classification_report,
    roc_auc_score,
    average_precision_score,
    make_scorer,
    matthews_corrcoef,
    f1_score,
    precision_score,
    recall_score,
    balanced_accuracy_score,
    accuracy_score,
    confusion_matrix,
    ConfusionMatrixDisplay,
    roc_curve,
    auc
)
```

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_validate
from sklearn.svm import SVC
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from xgboost import XGBClassifier
import matplotlib.pyplot as plt
import subprocess
import shap
import seaborn as sns
from sklearn.inspection import permutation_importance


warnings.filterwarnings("ignore")

print("Bibliotecas importadas")
```

```
Bibliotecas importadas
```

[2]:
```
'''
obtenção dos datasets
'''

EMAIL = "marcela.leite@gmail.com.br"
OUTPUT_DIR = "pipelineAD"
os.makedirs(OUTPUT_DIR, exist_ok=True)
Entrez.email = EMAIL

# ============================================================
# QUERIES
# ============================================================

# POSITIVE_QUERY = r'''
# ("Solanum lycopersicum"[Organism] OR "Nicotiana benthamiana"[Organism] OR␣
 ↪"Capsicum annuum"[Organism] OR "Arabidopsis thaliana"[Organism])
# AND( TMV OR ToMV OR ToBRFV OR PMMoV OR TMGMV OR PVY OR PepYMV OR TEV OR PVA␣
 ↪OR PVX OR TSWV OR GRSV  OR TCSV  OR CMV OR ToCV OR TICV OR AMV
# OR "virus resistance" OR "antiviral" OR "RNA silencing" OR "RDR" OR␣
 ↪"Dicer-like" OR "Argonaute" OR "AGO" OR "eIF4E" OR "R gene" OR "Sw-5" OR␣
 ↪"Tm-2" OR "RCY1"
# OR "Rx" OR "R protein"  OR "NLR" OR "NBS-LRR"  OR "NB-LRR" OR "TIR-NBS-LRR"␣
 ↪OR "CC-NBS-LRR" OR "RLK" OR "RLP" OR "LRR receptor kinase"
# OR "leucine rich repeat receptor kinase" OR "plant disease resistance␣
 ↪protein" OR "disease resistance protein" OR "RPM1" OR "RPS2" OR "RPS4" OR␣
 ↪"RPS5"
# OR "RPP" OR "SNC1"  OR "ADR1" OR "NDR1" OR "EDS1" OR "PAD4" )
```

```
# NOT ( partial OR fragment OR hypothetical OR predicted OR DAP-seq OR␣
 ↪"transcription factor" OR DNA-binding )
# '''

POSITIVE_QUERY = r'''
    ("Solanaceae"[Organism] NOT ("Solanum betaceum"[Organism] OR "Solanum␣
 ↪melongena"[Organism] OR "Solanum tuberosum"[Organism] OR "Physalis␣
 ↪peruviana"[Organism]))
    AND (TMV OR ToMV OR ToBRFV OR PMMoV OR TMGMV OR PVY OR PepYMV OR TEV OR PVA␣
 ↪OR PVX OR TSWV OR GRSV OR TCSV OR CMV OR ToCV OR TICV OR AMV
    OR "virus resistance" OR "antiviral" OR "RNA silencing" OR "RDR" OR␣
 ↪"Dicer-like" OR "Argonaute" OR "AGO" OR "eIF4E" OR "R gene" OR "Sw-5" OR␣
 ↪"Tm-2" OR "RCY1"
    OR "Rx" OR "R protein" OR "NLR" OR "NBS-LRR" OR "NB-LRR" OR "TIR-NBS-LRR"␣
 ↪OR "CC-NBS-LRR" OR "RLK" OR "RLP" OR "LRR receptor kinase"
    OR "leucine rich repeat receptor kinase" OR "plant disease resistance␣
 ↪protein" OR "disease resistance protein" OR "RPM1" OR "RPS2" OR "RPS4" OR␣
 ↪"RPS5"
    OR "RPP" OR "SNC1" OR "ADR1" OR "NDR1" OR "EDS1" OR "PAD4" OR "NB-ARC" OR␣
 ↪"NBS-ARC" OR "TIR domain"
    OR "PRR" OR "LYK" OR "LRR" OR "LECRK" OR "TNL" OR "NB-LRR" OR "IL-1R"
    OR "NB-ARC domain" OR "CNL Domain" OR "NBS-ARC" OR "leucine-rich repeat␣
 ↪domain" OR "TIR domain-containing" OR "LRR-containing" OR "coiled-coil"
    OR "leucine-rich repeat" OR "Kinase domain" OR "Nucleotide-binding site")
    NOT (partial OR fragment OR hypothetical OR predicted OR DAP-seq OR␣
 ↪"transcription factor" OR DNA-binding OR microtom )
'''
# #OR microtom
# NEGATIVE_QUERY = r'''
# ( "Solanum lycopersicum"[Organism] OR "Nicotiana benthamiana"[Organism] OR␣
 ↪"Capsicum annuum"[Organism] OR "Arabidopsis thaliana"[Organism])
# AND ( "actin" OR "tubulin" OR "ribosomal protein" OR "ATP synthase" OR␣
 ↪"photosystem" OR "elongation factor" OR "glycolysis" OR "metabolic enzyme" )
# NOT ( "virus resistance"  OR "antiviral" OR "RNA silencing" OR "RDR" OR␣
 ↪"Dicer-like" OR "Argonaute" OR "AGO" OR "eIF4E" OR "R gene" OR "Sw-5" OR␣
 ↪"Tm-2"
# OR "RCY1" OR "Rx" OR "R protein" OR "resistance" OR "NLR" OR "LRR" OR␣
 ↪"immune" OR "virus" )
# '''
NEGATIVE_QUERY = r'''
    ("Solanaceae"[Organism] NOT ("Solanum melongena"[Organism] OR "Solanum␣
 ↪tuberosum"[Organism] OR "Physalis peruviana"[Organism]))
    AND 100:3000[Sequence Length]
    NOT (TMV OR ToMV OR ToBRFV OR PMMoV OR TMGMV OR PVY OR PepYMV OR TEV OR PVA␣
 ↪OR PVX OR TSWV OR GRSV OR TCSV OR CMV OR ToCV OR TICV OR AMV
```

```
    OR "virus resistance" OR "antiviral" OR "RNA silencing" OR "RDR" OR␣
 ↪"Dicer-like" OR "Argonaute" OR "AGO" OR "eIF4E" OR "R gene" OR "Sw-5" OR␣
 ↪"Tm-2" OR "RCY1"
    OR "Rx" OR "R protein" OR "NLR" OR "NBS-LRR" OR "NB-LRR" OR "TIR-NBS-LRR"␣
 ↪OR "CC-NBS-LRR" OR "RLK" OR "RLP" OR "LRR receptor kinase"
    OR "leucine rich repeat receptor kinase" OR "plant disease resistance␣
 ↪protein" OR "disease resistance protein" OR "RPM1" OR "RPS2" OR "RPS4" OR␣
 ↪"RPS5"
    OR "PRR" OR "LYK" OR "LRR" OR "LECRK" OR "TNL" OR "NB-LRR" OR "IL-1R" OR␣
 ↪"NB-ARC" OR "NB-ARC domain"
    OR "RPP" OR "SNC1" OR "ADR1" OR "NDR1" OR "EDS1" OR "PAD4" OR "coiled-coil"␣
 ↪ OR "leucine-rich repeat" OR "Kinase domain" OR "Nucleotide-binding site")
    NOT (partial OR fragment OR hypothetical OR predicted OR uncharacterized OR␣
 ↪DAP-seq OR "transcription factor" OR DNA-binding OR microtom )
'''

# Dados dos transcriptomas para a aplicação - previsão de genes de resistência
# ARABIDOPSIS_QUERY = '''
#   "Arabidopsis thaliana"[Organism] AND (transcriptome OR RNA-Seq OR TSA OR␣
 ↪mRNA)
#       NOT (genome OR chromosome OR scaffold OR contig OR partial OR fragment␣
 ↪OR microtom)
#       AND 1500:30000[Sequence Length]
#       '''
ARABIDOPSIS_QUERY = '''
   "Arabidopsis thaliana"[Organism] AND (biomol_mrna[PROP] OR tsa[keyword])
      AND 1500:30000[Sequence Length]
'''

SOLANUM_MELONGENA = '''
    "Solanum melongena"[Organism] AND (biomol_mrna[PROP] OR tsa[keyword])
      AND 1500:30000[Sequence Length]
    '''

TUBEROSUM_QUERY = '''
    "Solanum tuberosum"[Organism] AND (biomol_mrna[PROP] OR tsa[keyword])
      AND 1500:30000[Sequence Length]
    '''

PHYSALIS_QUERY = '''
  "Physalis peruviana"[Organism] AND (biomol_mrna[PROP] OR tsa[keyword])
      AND 1500:30000[Sequence Length]
      '''

# ============================================================
# DOWNLOAD NCBI
```

```python
# ============================================================

def baixar_proteinas_ncbi(query, output_fasta, max_records=35000):
    print("\n================================")
    print("DOWNLOAD NCBI: ",output_fasta)
    print("================================")
    handle = Entrez.esearch(
        db="protein",
        term=query,
        retmax=max_records
    )
    record = Entrez.read(handle)
    ids = record["IdList"]
    print(f"Proteínas encontradas: {len(ids)}")
    if len(ids) == 0:
        return
    handle = Entrez.efetch(
        db="protein",
        id=ids,
        rettype="fasta",
        retmode="text"
    )
    with open(output_fasta, "w") as f:
        f.write(handle.read())
    print(f"Arquivo salvo: {output_fasta}")

def baixar_transcriptoma(output_fasta, query):
    print("\n================================")
    print("DOWNLOAD TRANSCRIPTOMA: ",output_fasta)
    print("================================")
    handle = Entrez.esearch(
        db="nucleotide",
        term=query,
        retmax=50000
    )
    record = Entrez.read(handle)
    ids = record["IdList"]
    print(f"Transcritos encontrados: {len(ids)}")
    handle = Entrez.efetch(
        db="nucleotide",
        id=ids,
        rettype="fasta",
        retmode="text"
    )
    with open(output_fasta, "w") as f:
        f.write(handle.read())
    print(f"Arquivo salvo: {output_fasta}")
```

```
#Dados de Entrada - baixar arquivos FASTA das sequências de proteínas
POSITIVE_FASTA = f"{OUTPUT_DIR}/positive.fasta"
NEGATIVE_FASTA = f"{OUTPUT_DIR}/negative.fasta"


print("\n============ FINALIZADO CONFIGURAÇÃO DE PARÂMETROS ============\n")
```

```
============ FINALIZADO CONFIGURAÇÃO DE PARÂMETROS ============
```

[3]:
```
inicio = datetime.now()
print("\n\nInciando download da classe positiva: ", inicio.strftime("%H:%M:%S"))
baixar_proteinas_ncbi(POSITIVE_QUERY,POSITIVE_FASTA)
fim = datetime.now()
print("\n\nFinalizado em:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)
```

```
Inciando download da classe positiva:  15:26:15

================================
DOWNLOAD NCBI:  pipelineAD/positive.fasta
================================
Proteínas encontradas: 34655
Arquivo salvo: pipelineAD/positive.fasta


Finalizado em: 15:27:19

Tempo decorrido: 0:01:04.388176
```

[4]:
```
inicio = datetime.now()
print("\n\nInciando download da classe negativa: ", inicio.strftime("%H:%M:%S"))
baixar_proteinas_ncbi(NEGATIVE_QUERY,NEGATIVE_FASTA)
fim = datetime.now()
print("\n\nFinalizado em:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)

print("\n\nArquivos salvos na pasta: ",OUTPUT_DIR)
print("============ FIM COLETA DE DADOS TREINO/VALIDAÇÃO ============")
```

```
Inciando download da classe negativa:  15:27:19

================================
```

```
DOWNLOAD NCBI:  pipelineAD/negative.fasta
================================
Proteínas encontradas: 35000
Arquivo salvo: pipelineAD/negative.fasta


Finalizado em: 15:28:16

Tempo decorrido: 0:00:56.094919


Arquivos salvos na pasta:  pipelineAD
============ FIM COLETA DE DADOS TREINO/VALIDAÇÃO ============
```

[3]:
```
#Dados para aplicação - baixar transcriptomas - RNA-Seq
transcriptoma_arabidopsis_fasta = (f"{OUTPUT_DIR}/arabidopsis_transcriptoma.
  ↪fasta")
transcriptoma_melongena_fasta = (f"{OUTPUT_DIR}/melongena_transcriptoma.fasta")
transcriptoma_tuberosum_fasta = (f"{OUTPUT_DIR}/tuberosum_transcriptoma.fasta")
transcriptoma_physalis_fasta = (f"{OUTPUT_DIR}/physalis_transcriptoma.fasta")
```

[6]:
```
print("\n\nInciando downaload de dados de transcriptmas")
inicio = datetime.now()
print("\n\nInciando download ARABIDOPSIS: ", inicio.strftime("%H:%M:%S"))
baixar_transcriptoma(transcriptoma_arabidopsis_fasta, ARABIDOPSIS_QUERY)
fim = datetime.now()
print("\n\nFinalizado em:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)
```

```
Inciando downaload de dados de transcriptmas


Inciando download ARABIDOPSIS:  15:28:16

================================
DOWNLOAD TRANSCRIPTOMA:  pipelineAD/arabidopsis_transcriptoma.fasta
================================
Transcritos encontrados: 49951
Arquivo salvo: pipelineAD/arabidopsis_transcriptoma.fasta


Finalizado em: 15:29:00

Tempo decorrido: 0:00:44.451853
```

[7]:
```
inicio = datetime.now()
print("\n\nInciando download PHYSALIS: ", inicio.strftime("%H:%M:%S"))
baixar_transcriptoma(transcriptoma_physalis_fasta, PHYSALIS_QUERY)
fim = datetime.now()
print("\n\nFinalizado em:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)
```

```
Inciando download PHYSALIS:  15:29:00

================================
DOWNLOAD TRANSCRIPTOMA:  pipelineAD/physalis_transcriptoma.fasta
================================
Transcritos encontrados: 21702
Arquivo salvo: pipelineAD/physalis_transcriptoma.fasta


Finalizado em: 15:29:43

Tempo decorrido: 0:00:42.981751
```

[8]:
```
inicio = datetime.now()
print("\n\nInciando download MELONGENA: ", inicio.strftime("%H:%M:%S"))
baixar_transcriptoma(transcriptoma_melongena_fasta, SOLANUM_MELONGENA)
fim = datetime.now()
print("\n\nFinalizado em:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)
```

```
Inciando download MELONGENA:  15:29:43

================================
DOWNLOAD TRANSCRIPTOMA:  pipelineAD/melongena_transcriptoma.fasta
================================
Transcritos encontrados: 8555
Arquivo salvo: pipelineAD/melongena_transcriptoma.fasta


Finalizado em: 15:30:19

Tempo decorrido: 0:00:36.107486
```

[9]:
```
inicio = datetime.now()
print("\n\nInciando download TUBEROSUM: ", inicio.strftime("%H:%M:%S"))
baixar_transcriptoma(transcriptoma_tuberosum_fasta, TUBEROSUM_QUERY)
fim = datetime.now()
```

```
print("\n\nFinalizado em:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)

print("\n\n\nArquivos salvos na pasta: ",OUTPUT_DIR)
print("============ FIM COLETA DE DADOS APLICAÇÃO ============")
```

```
Inciando download TUBEROSUM:  15:30:19

================================
DOWNLOAD TRANSCRIPTOMA:  pipelineAD/tuberosum_transcriptoma.fasta
================================
Transcritos encontrados: 20832
Arquivo salvo: pipelineAD/tuberosum_transcriptoma.fasta


Finalizado em: 15:31:31

Tempo decorrido: 0:01:11.517551



Arquivos salvos na pasta:  pipelineAD
============ FIM COLETA DE DADOS APLICAÇÃO ============
```

[4]:
```
'''
Tratamento dos dados
    - Criar Dataframes com os conjuntos de dados de treino e validação
    - Executar CD-Hit para eliminar sequencias muitos semelhantes
    - Extração de atributos (features)

'''

# ============================================================
# AAC
# ============================================================
# composição de aminoácidos - transformar a proteína em dados numéricos
# conta a frequência de cada aminoácido em uma sequência de proteína
# como são 20 aminoácidos possíveis, irá gerar 20 features com seus percentuais
# com essa frequência consegue ter um parâmetro para distinguir as proteínas
# por exemplo proteínas de resistência tem geralmente certa frequência de␣
 ↪aminoácidos
# mas ignora a ordem - por isso a sequência de dipeptídeos irá incrementar o␣
 ↪modelo considerando a ordem
# sequência de 20 aminoácidos
AMINO_ACIDOS = list("ACDEFGHIKLMNPQRSTVWY")
```

```
motifs = {
    # ============================
    # NLR (CNL/TNL)
    # ============================
    "P_LOOP": r"[AGST].{3,5}GK[TS]",
    "KINASE2": r"[ALIVMFYW]{3,5}DD[VILW][WFY]",
    "GLPL": r"G[LVI][PAST][LIVAF]",
    "MHD": r"[MVIL][HN][DE][VILAST]",
    "RNBS_A": r"F.DL.{0,3}AW",
    "RNBS_B": r"G[SA].{2,6}T",
    "RNBS_C": r"Y.[VIL].{1,3}[LS]",
    "RNBS_D": r"C.{2,8}P",

    # ============================
    # Protein Kinase (KIN/RLK)
    # ============================
    "HRD": r"HRD[LIVMFY]",
    "DFG": r"DFG",

    # ============================
    # LRR (RLP/RLK)
    # ============================
    "LRR1": r"L..L..L..N",
    "LRR2": r"L[A-Z]{2}L[A-Z]{2}L[A-Z]{2}[NCT]"

}

def calcular_aac(seq):
    seq = seq.upper()
    total = len(seq)
    if total == 0:
        return [0] * len(AMINO_ACIDOS)
    counts = Counter(seq)
    return [
        counts.get(aa, 0) / total
        for aa in AMINO_ACIDOS
    ]

# ============================================================
# DIPEPTÍDEOS
# ============================================================
'''
encontrar pares de aminoácidos consectíveis em uma sequência protéica ␣
 ↪-sequência de dois aminoácidos
considera a ordem dos aminoácidos
há certos padrões comuns para
```

```
para 20 aminoácidos, há possibilidade de 20*20 = 400 dipeptídeos - features
'''

DIPEPTIDES = [
    a + b
    for a in AMINO_ACIDOS
    for b in AMINO_ACIDOS
]

def calcular_dipeptideos(seq):
    seq = seq.upper()
    total = len(seq) - 1
    if total <= 0:
        return [0] * len(DIPEPTIDES)
    counts = Counter([
        seq[i:i+2]
        for i in range(total)
    ])
    return [
        counts.get(dp, 0) / total
        for dp in DIPEPTIDES
    ]

def localizar_motivos(seq):
    posicoes = {}
    for nome, pattern in motifs.items():
        match = re.search(pattern, seq)

        if match:
            posicoes[nome] = match.start()
        else:
            posicoes[nome] = None

    return posicoes


def calcular_distancias(posicoes):

    def distancia(a, b):
        if posicoes[a] is None or posicoes[b] is None:
            return -1
        return posicoes[b] - posicoes[a]

    return {
        "dist_ploop_kinase2": distancia("P_LOOP", "KINASE2"),
        "dist_kinase2_glpl": distancia("KINASE2", "GLPL"),
        "dist_glpl_mhd": distancia("GLPL", "MHD")
```

```
    }

def contar_motivos(posicoes):
    return sum(
        1 for v in posicoes.values()
        if v is not None
    )

def extrair_motifs(seq):

    return [
        int(bool(re.search(pattern, seq)))
        for pattern in motifs.values()
    ]



# ============================================================
# FEATURES - atributos
# ============================================================
#calculando features
def extrair_features_proteina(seq):
    seq = re.sub(r"[^ACDEFGHIKLMNPQRSTVWY]", "", seq)
    if len(seq) < 50:
        return None
    features = []
    features.append(len(seq)) # 1 - comprimento da cadeia
    features.extend(calcular_aac(seq)) # 20 composição de aminoácidos -␣
 ↪frequência de cada um na sequência
    features.extend(calcular_dipeptideos(seq)) #400 possibilidades - considera␣
 ↪a ordem dos aminoácidos para buscar os mais frequêntes em proteínas de␣
 ↪resistência

    features.extend(extrair_motifs(seq))
    # posições dos motivos
    posicoes = localizar_motivos(seq)
    # número de motivos encontrados
    num_motifs_detectados = contar_motivos(posicoes)
    # distâncias entre motivos
    distancias = calcular_distancias(posicoes)

    features.append(num_motifs_detectados)
    protein_length = len(seq)

    features.append(distancias["dist_ploop_kinase2"])
    features.append(distancias["dist_kinase2_glpl"])
    features.append(distancias["dist_glpl_mhd"])
```

```
features.append(
    distancias["dist_ploop_kinase2"]/protein_length
    if distancias["dist_ploop_kinase2"] != -1 else -1
)
features.append(
    distancias["dist_kinase2_glpl"]/protein_length
    if distancias["dist_kinase2_glpl"] != -1 else -1
)
features.append(
    distancias["dist_glpl_mhd"]/protein_length
    if distancias["dist_glpl_mhd"] != -1 else -1
)

features.append(int(posicoes["P_LOOP"] is not None))
features.append(int(posicoes["KINASE2"] is not None))
features.append(int(posicoes["GLPL"] is not None))
features.append(int(posicoes["MHD"] is not None))

encontrados = {}
for nome, regex in motifs.items():
    encontrados[nome] = int( re.search(regex, seq) is not None)

# features.append(encontrados[nome])

num_nlr = (
    encontrados["P_LOOP"] +
    encontrados["KINASE2"] +
    encontrados["GLPL"] +
    encontrados["MHD"]
)

# num_kinase = (
#     encontrados["HRD"] +
#     encontrados["DFG"]
# )

# num_lrr = (
#     encontrados["LRR1"] +
#     encontrados["LRR2"]
# )

features.extend([
    num_nlr,
    # num_kinase,
    # num_lrr
])
```

```
    return features

# ============================================================
# DATASET
# ============================================================
motif_cols = [
    "P_LOOP",
    "KINASE2",
    "GLPL",
    "MHD",
    "RNBS_A",
    "RNBS_B",
    "RNBS_C",
    "RNBS_D",
    "HRD",
    "DFG",
    "LRR1",
    "LRR2",
    "NUM_NLR_MOTIFS"
 ]
    # "NUM_KINASE_MOTIFS",
    # "NUM_LRR_MOTIFS"

def criar_dataset(positive_fasta, negative_fasta):
    data = []
    columns = (
        ["protein_length"]
        + [f"AAC_{aa}" for aa in AMINO_ACIDOS]
        + [f"DIPEP_{dp}" for dp in DIPEPTIDES]
        + motif_cols
        + ["num_motifs_detectados"]
        + ["dist_ploop_kinase2"]
        + ["dist_kinase2_glpl"]
        + ["dist_glpl_mhd"]
        + ["dist_ploop_kinase2_norm"]
        + ["dist_kinase2_glpl_norm"]
        + ["dist_glpl_mhd_norm"]
        + ["has_ploop"]
        + ["has_kinase2"]
        + ["has_glpl"]
        + ["has_mhd"]
    )

    # POSITIVOS
    for record in SeqIO.parse(positive_fasta, "fasta"):
        seq = str(record.seq)
```

```
        feats = extrair_features_proteina(seq)
        if feats is None:
            continue
        row = feats + [1, record.id]
        data.append(row)

    # NEGATIVOS
    for record in SeqIO.parse(negative_fasta, "fasta"):
        seq = str(record.seq)
        feats = extrair_features_proteina(seq)
        if feats is None:
            continue
        row = feats + [0, record.id]
        data.append(row)

    df = pd.DataFrame(
        data,
        columns=columns + ["target", "id"]
    )
    print("\nTotal de Features/Atributos:",len(columns)) # quantidade
    print("\nFeatures/atributos:")
    print(columns)
    return df, columns

# CD-Hit
# chama o programa CD-HIT do sistema operacional
def executar_cdhit(
    input_fasta,
    output_fasta,
    identity=0.7
):
    cmd = [
        "cd-hit",
        "-i", input_fasta,
        "-o", output_fasta,
        "-c", str(identity),
        "-n", "5",
        "-M", "30000",
        "-T", "4"
    ]

    print("\nExecutando CD-HIT...")
    subprocess.run(cmd, check=True)
    print("CD-HIT concluído.")

def limpar(seq_id):
    seq_id = seq_id.strip()
```

```
    seq_id = seq_id.split()[0]
    return seq_id

def ler_clusters_cdhit(cluster_file):
    clusters = {}
    cluster_id = None
    with open(cluster_file) as f:
        for line in f:
            line = line.strip()
            if line.startswith(">Cluster"):
                cluster_id = line.replace(">Cluster ", "")
                clusters[cluster_id] = []
            else:
                seq_id = line.split(">")[1].split("...")[0]
                seq_id = limpar(seq_id)
                clusters[cluster_id].append(seq_id)
    return clusters

inicio = datetime.now()
print("\n\n\nInciando Tratamento dos dados: ", inicio.strftime("%H:%M:%S"))

# executa o CD-Hit nos arquivos baixados
print("\n\nInciar a execução do CD-HIT\n")
executar_cdhit(
    POSITIVE_FASTA,
    f"{OUTPUT_DIR}/positive_nr.fasta",
    identity=0.7
)

executar_cdhit(
    NEGATIVE_FASTA,
    f"{OUTPUT_DIR}/negative_nr.fasta",
    identity=0.7
)
# ========================================================
# Montar os DATASETs
# ========================================================

POSITIVE_FASTA = f"{OUTPUT_DIR}/positive_nr.fasta"
NEGATIVE_FASTA = f"{OUTPUT_DIR}/negative_nr.fasta"


df, columns = criar_dataset(
    POSITIVE_FASTA,
    NEGATIVE_FASTA
)
```

```
df["id"] = df["id"].apply(limpar)
# ======================================================
# CLUSTERS
# ======================================================

clusters_pos = ler_clusters_cdhit(
    f"{OUTPUT_DIR}/positive_nr.fasta.clstr"
)

clusters_neg = ler_clusters_cdhit(
    f"{OUTPUT_DIR}/negative_nr.fasta.clstr"
)

cluster_map = {}

for c, seqs in clusters_pos.items():
    for s in seqs:
        cluster_map[s] = f"POS_{c}"

for c, seqs in clusters_neg.items():
    for s in seqs:
        cluster_map[s] = f"NEG_{c}"

df["cluster"] = df["id"].map(cluster_map)

print("\n Clusters ausentes: ")
print(df["cluster"].isna().sum())
df = df.dropna(subset=["cluster"])

print("\nDataset original pós CD-HIT:", df.shape)

# ======================================================
# NOVO BLOCO: BALANCEAMENTO IMPERATIVO (DOWNSAMPLING)
# ======================================================
print("\n--- Executando Balanceamento de Classes ---")
df_positivos = df[df["target"] == 1]
df_negativos = df[df["target"] == 0]

print(f"Contagem inicial -> Positivos (R-genes): {len(df_positivos)} |␣
 ↪Negativos (Background): {len(df_negativos)}")

# Se houver mais negativos do que positivos (cenário padrão), reduzimos os␣
 ↪negativos
if len(df_negativos) > len(df_positivos):
    # O sample() escolhe aleatoriamente N linhas do grupo majoritário
    df_negativos_balanceados = df_negativos.sample(n=len(df_positivos),␣
 ↪random_state=42)
```

```
    df = pd.concat([df_positivos, df_negativos_balanceados]).
 ↪reset_index(drop=True)
    print(f"Subamostragem aplicada com sucesso nos Negativos.")
else:
    print("O dataset já está balanceado ou possui mais positivos que negativos␣
 ↪(incomum). Nenhuma ação foi tomada.")

print(f"Contagem final   -> Positivos: {len(df[df['target'] == 1])} | Negativos:
 ↪ {len(df[df['target'] == 0])}")
print("Dataset Final Pronto:", df.shape)


fim = datetime.now()
print("\n\nFinalizado em:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)
print("\n\nFim tratamento dos dados...")
```

```
Inciando Tratamento dos dados:  15:35:53


Inciar a execução do CD-HIT


Executando CD-HIT…
================================================================
Program: CD-HIT, V4.8.1 (+OpenMP), Aug 20 2021, 08:39:56
Command: cd-hit -i pipelineAD/positive.fasta -o
         pipelineAD/positive_nr.fasta -c 0.7 -n 5 -M 30000 -T 4

Started: Fri Jun 12 15:35:53 2026
================================================================
                            Output
----------------------------------------------------------------
total seq: 9999
longest and shortest : 3540 and 28
Total letters: 7105544
Sequences have been sorted

Approximated minimal memory consumption:
Sequence        : 8M
Buffer          : 4 X 11M = 45M
Table           : 2 X 65M = 131M
Miscellaneous   : 0M
Total           : 184M
```

```
Table limit with the given memory limit:
Max number of representatives: 4000000
Max number of word counting entries: 3726877013

# comparing sequences from          0  to       1666
.---------- new table with      366 representatives
# comparing sequences from       1666  to       3054
----------    595 remaining sequences to the next cycle
---------- new table with      187 representatives
# comparing sequences from       2459  to       3715
----------    743 remaining sequences to the next cycle
---------- new table with      123 representatives
# comparing sequences from       2972  to       4143
----------    656 remaining sequences to the next cycle
---------- new table with      100 representatives
# comparing sequences from       3487  to       4572
----------    663 remaining sequences to the next cycle
---------- new table with      101 representatives
# comparing sequences from       3909  to       4924
----------    627 remaining sequences to the next cycle
---------- new table with      100 representatives
# comparing sequences from       4297  to       5247
----------    538 remaining sequences to the next cycle
---------- new table with      100 representatives
# comparing sequences from       4709  to       5590
----------    371 remaining sequences to the next cycle
---------- new table with      100 representatives
# comparing sequences from       5219  to       6015
----------    181 remaining sequences to the next cycle
---------- new table with      100 representatives
# comparing sequences from       5834  to       6528
----------    303 remaining sequences to the next cycle
---------- new table with      100 representatives
# comparing sequences from       6225  to       6854
----------    301 remaining sequences to the next cycle
---------- new table with      100 representatives
# comparing sequences from       6553  to       7127
----------    249 remaining sequences to the next cycle
---------- new table with      100 representatives
# comparing sequences from       6878  to       7398
----------    222 remaining sequences to the next cycle
---------- new table with      100 representatives
# comparing sequences from       7176  to       7646
----------    126 remaining sequences to the next cycle
---------- new table with      100 representatives
# comparing sequences from       7520  to       7933
----------     29 remaining sequences to the next cycle
```

```
---------- new table with      100 representatives
# comparing sequences from       7904  to       8253
…---------- new table with       93 representatives
# comparing sequences from       8253  to       8544
…---------- new table with       94 representatives
# comparing sequences from       8544  to       8786
…---------- new table with       83 representatives
# comparing sequences from       8786  to       8988
100.0%---------- new table with        5 representatives
# comparing sequences from       8988  to       9156
…---------- new table with       49 representatives
# comparing sequences from       9156  to       9999
…---------- new table with      179 representatives

     9999  finished       2380  clusters

Approximated maximum memory consumption: 190M
writing new database
writing clustering information
program completed !

Total CPU time 2.79
CD-HIT concluído.

Executando CD-HIT…
================================================================
Program: CD-HIT, V4.8.1 (+OpenMP), Aug 20 2021, 08:39:56
Command: cd-hit -i pipelineAD/negative.fasta -o
         pipelineAD/negative_nr.fasta -c 0.7 -n 5 -M 30000 -T 4

Started: Fri Jun 12 15:35:54 2026
================================================================
                            Output
----------------------------------------------------------------
total seq: 9999
longest and shortest : 2993 and 100
Total letters: 4226744
Sequences have been sorted

Approximated minimal memory consumption:
Sequence        : 5M
Buffer          : 4 X 11M = 44M
Table           : 2 X 65M = 131M
Miscellaneous   : 0M
Total           : 181M

Table limit with the given memory limit:
Max number of representatives: 4000000
```

```
Max number of word counting entries: 3727296166

# comparing sequences from          0  to       1666
.---------- new table with     1378 representatives
# comparing sequences from       1666  to       3054
----------    672 remaining sequences to the next cycle
---------- new table with      592 representatives
# comparing sequences from       2382  to       3651
----------    803 remaining sequences to the next cycle
---------- new table with      301 representatives
# comparing sequences from       2848  to       4039
----------    811 remaining sequences to the next cycle
---------- new table with      260 representatives
# comparing sequences from       3228  to       4356
----------    795 remaining sequences to the next cycle
---------- new table with      278 representatives
# comparing sequences from       3561  to       4634
----------    737 remaining sequences to the next cycle
---------- new table with      284 representatives
# comparing sequences from       3897  to       4914
----------    707 remaining sequences to the next cycle
---------- new table with      246 representatives
# comparing sequences from       4207  to       5172
----------    668 remaining sequences to the next cycle
---------- new table with      217 representatives
# comparing sequences from       4504  to       5419
----------    646 remaining sequences to the next cycle
---------- new table with      229 representatives
# comparing sequences from       4773  to       5644
----------    556 remaining sequences to the next cycle
---------- new table with      202 representatives
# comparing sequences from       5088  to       5906
----------    545 remaining sequences to the next cycle
---------- new table with      205 representatives
# comparing sequences from       5361  to       6134
----------    529 remaining sequences to the next cycle
---------- new table with      198 representatives
# comparing sequences from       5605  to       6337
----------    497 remaining sequences to the next cycle
---------- new table with      156 representatives
# comparing sequences from       5840  to       6533
----------    463 remaining sequences to the next cycle
---------- new table with      175 representatives
# comparing sequences from       6070  to       6724
----------    427 remaining sequences to the next cycle
---------- new table with      173 representatives
# comparing sequences from       6297  to       6914
----------    425 remaining sequences to the next cycle
```

```
---------- new table with      131 representatives
# comparing sequences from       6489  to       7074
----------    390 remaining sequences to the next cycle
---------- new table with      128 representatives
# comparing sequences from       6684  to       7236
----------    362 remaining sequences to the next cycle
---------- new table with      142 representatives
# comparing sequences from       6874  to       7394
----------    338 remaining sequences to the next cycle
---------- new table with      119 representatives
# comparing sequences from       7056  to       7546
----------    321 remaining sequences to the next cycle
---------- new table with      118 representatives
# comparing sequences from       7225  to       7687
----------    293 remaining sequences to the next cycle
---------- new table with      130 representatives
# comparing sequences from       7394  to       7828
----------    257 remaining sequences to the next cycle
---------- new table with      111 representatives
# comparing sequences from       7571  to       7975
----------    250 remaining sequences to the next cycle
---------- new table with      119 representatives
# comparing sequences from       7725  to       8104
----------    215 remaining sequences to the next cycle
---------- new table with      100 representatives
# comparing sequences from       7889  to       8240
----------    204 remaining sequences to the next cycle
---------- new table with      100 representatives
# comparing sequences from       8036  to       8363
----------    195 remaining sequences to the next cycle
---------- new table with      100 representatives
# comparing sequences from       8168  to       8473
----------    162 remaining sequences to the next cycle
---------- new table with      100 representatives
# comparing sequences from       8311  to       8592
----------    127 remaining sequences to the next cycle
---------- new table with      100 representatives
# comparing sequences from       8465  to       8720
----------     78 remaining sequences to the next cycle
---------- new table with      100 representatives
# comparing sequences from       8642  to       8868
----------     33 remaining sequences to the next cycle
---------- new table with      100 representatives
# comparing sequences from       8835  to       9029
----------     40 remaining sequences to the next cycle
---------- new table with      100 representatives
# comparing sequences from       8989  to       9157
…---------- new table with       88 representatives
```

```
# comparing sequences from       9157  to       9999
…---------- new table with      404 representatives

     9999  finished       7184  clusters

Approximated maximum memory consumption: 194M
writing new database
writing clustering information
program completed !

Total CPU time 1.71
CD-HIT concluído.

Total de Features/Atributos: 445

Features/atributos:
['protein_length', 'AAC_A', 'AAC_C', 'AAC_D', 'AAC_E', 'AAC_F', 'AAC_G',
'AAC_H', 'AAC_I', 'AAC_K', 'AAC_L', 'AAC_M', 'AAC_N', 'AAC_P', 'AAC_Q', 'AAC_R',
'AAC_S', 'AAC_T', 'AAC_V', 'AAC_W', 'AAC_Y', 'DIPEP_AA', 'DIPEP_AC', 'DIPEP_AD',
'DIPEP_AE', 'DIPEP_AF', 'DIPEP_AG', 'DIPEP_AH', 'DIPEP_AI', 'DIPEP_AK',
'DIPEP_AL', 'DIPEP_AM', 'DIPEP_AN', 'DIPEP_AP', 'DIPEP_AQ', 'DIPEP_AR',
'DIPEP_AS', 'DIPEP_AT', 'DIPEP_AV', 'DIPEP_AW', 'DIPEP_AY', 'DIPEP_CA',
'DIPEP_CC', 'DIPEP_CD', 'DIPEP_CE', 'DIPEP_CF', 'DIPEP_CG', 'DIPEP_CH',
'DIPEP_CI', 'DIPEP_CK', 'DIPEP_CL', 'DIPEP_CM', 'DIPEP_CN', 'DIPEP_CP',
'DIPEP_CQ', 'DIPEP_CR', 'DIPEP_CS', 'DIPEP_CT', 'DIPEP_CV', 'DIPEP_CW',
'DIPEP_CY', 'DIPEP_DA', 'DIPEP_DC', 'DIPEP_DD', 'DIPEP_DE', 'DIPEP_DF',
'DIPEP_DG', 'DIPEP_DH', 'DIPEP_DI', 'DIPEP_DK', 'DIPEP_DL', 'DIPEP_DM',
'DIPEP_DN', 'DIPEP_DP', 'DIPEP_DQ', 'DIPEP_DR', 'DIPEP_DS', 'DIPEP_DT',
'DIPEP_DV', 'DIPEP_DW', 'DIPEP_DY', 'DIPEP_EA', 'DIPEP_EC', 'DIPEP_ED',
'DIPEP_EE', 'DIPEP_EF', 'DIPEP_EG', 'DIPEP_EH', 'DIPEP_EI', 'DIPEP_EK',
'DIPEP_EL', 'DIPEP_EM', 'DIPEP_EN', 'DIPEP_EP', 'DIPEP_EQ', 'DIPEP_ER',
'DIPEP_ES', 'DIPEP_ET', 'DIPEP_EV', 'DIPEP_EW', 'DIPEP_EY', 'DIPEP_FA',
'DIPEP_FC', 'DIPEP_FD', 'DIPEP_FE', 'DIPEP_FF', 'DIPEP_FG', 'DIPEP_FH',
'DIPEP_FI', 'DIPEP_FK', 'DIPEP_FL', 'DIPEP_FM', 'DIPEP_FN', 'DIPEP_FP',
'DIPEP_FQ', 'DIPEP_FR', 'DIPEP_FS', 'DIPEP_FT', 'DIPEP_FV', 'DIPEP_FW',
'DIPEP_FY', 'DIPEP_GA', 'DIPEP_GC', 'DIPEP_GD', 'DIPEP_GE', 'DIPEP_GF',
'DIPEP_GG', 'DIPEP_GH', 'DIPEP_GI', 'DIPEP_GK', 'DIPEP_GL', 'DIPEP_GM',
'DIPEP_GN', 'DIPEP_GP', 'DIPEP_GQ', 'DIPEP_GR', 'DIPEP_GS', 'DIPEP_GT',
'DIPEP_GV', 'DIPEP_GW', 'DIPEP_GY', 'DIPEP_HA', 'DIPEP_HC', 'DIPEP_HD',
'DIPEP_HE', 'DIPEP_HF', 'DIPEP_HG', 'DIPEP_HH', 'DIPEP_HI', 'DIPEP_HK',
'DIPEP_HL', 'DIPEP_HM', 'DIPEP_HN', 'DIPEP_HP', 'DIPEP_HQ', 'DIPEP_HR',
'DIPEP_HS', 'DIPEP_HT', 'DIPEP_HV', 'DIPEP_HW', 'DIPEP_HY', 'DIPEP_IA',
'DIPEP_IC', 'DIPEP_ID', 'DIPEP_IE', 'DIPEP_IF', 'DIPEP_IG', 'DIPEP_IH',
'DIPEP_II', 'DIPEP_IK', 'DIPEP_IL', 'DIPEP_IM', 'DIPEP_IN', 'DIPEP_IP',
'DIPEP_IQ', 'DIPEP_IR', 'DIPEP_IS', 'DIPEP_IT', 'DIPEP_IV', 'DIPEP_IW',
'DIPEP_IY', 'DIPEP_KA', 'DIPEP_KC', 'DIPEP_KD', 'DIPEP_KE', 'DIPEP_KF',
'DIPEP_KG', 'DIPEP_KH', 'DIPEP_KI', 'DIPEP_KK', 'DIPEP_KL', 'DIPEP_KM',
'DIPEP_KN', 'DIPEP_KP', 'DIPEP_KQ', 'DIPEP_KR', 'DIPEP_KS', 'DIPEP_KT',
```

```
'DIPEP_KV', 'DIPEP_KW', 'DIPEP_KY', 'DIPEP_LA', 'DIPEP_LC', 'DIPEP_LD',
'DIPEP_LE', 'DIPEP_LF', 'DIPEP_LG', 'DIPEP_LH', 'DIPEP_LI', 'DIPEP_LK',
'DIPEP_LL', 'DIPEP_LM', 'DIPEP_LN', 'DIPEP_LP', 'DIPEP_LQ', 'DIPEP_LR',
'DIPEP_LS', 'DIPEP_LT', 'DIPEP_LV', 'DIPEP_LW', 'DIPEP_LY', 'DIPEP_MA',
'DIPEP_MC', 'DIPEP_MD', 'DIPEP_ME', 'DIPEP_MF', 'DIPEP_MG', 'DIPEP_MH',
'DIPEP_MI', 'DIPEP_MK', 'DIPEP_ML', 'DIPEP_MM', 'DIPEP_MN', 'DIPEP_MP',
'DIPEP_MQ', 'DIPEP_MR', 'DIPEP_MS', 'DIPEP_MT', 'DIPEP_MV', 'DIPEP_MW',
'DIPEP_MY', 'DIPEP_NA', 'DIPEP_NC', 'DIPEP_ND', 'DIPEP_NE', 'DIPEP_NF',
'DIPEP_NG', 'DIPEP_NH', 'DIPEP_NI', 'DIPEP_NK', 'DIPEP_NL', 'DIPEP_NM',
'DIPEP_NN', 'DIPEP_NP', 'DIPEP_NQ', 'DIPEP_NR', 'DIPEP_NS', 'DIPEP_NT',
'DIPEP_NV', 'DIPEP_NW', 'DIPEP_NY', 'DIPEP_PA', 'DIPEP_PC', 'DIPEP_PD',
'DIPEP_PE', 'DIPEP_PF', 'DIPEP_PG', 'DIPEP_PH', 'DIPEP_PI', 'DIPEP_PK',
'DIPEP_PL', 'DIPEP_PM', 'DIPEP_PN', 'DIPEP_PP', 'DIPEP_PQ', 'DIPEP_PR',
'DIPEP_PS', 'DIPEP_PT', 'DIPEP_PV', 'DIPEP_PW', 'DIPEP_PY', 'DIPEP_QA',
'DIPEP_QC', 'DIPEP_QD', 'DIPEP_QE', 'DIPEP_QF', 'DIPEP_QG', 'DIPEP_QH',
'DIPEP_QI', 'DIPEP_QK', 'DIPEP_QL', 'DIPEP_QM', 'DIPEP_QN', 'DIPEP_QP',
'DIPEP_QQ', 'DIPEP_QR', 'DIPEP_QS', 'DIPEP_QT', 'DIPEP_QV', 'DIPEP_QW',
'DIPEP_QY', 'DIPEP_RA', 'DIPEP_RC', 'DIPEP_RD', 'DIPEP_RE', 'DIPEP_RF',
'DIPEP_RG', 'DIPEP_RH', 'DIPEP_RI', 'DIPEP_RK', 'DIPEP_RL', 'DIPEP_RM',
'DIPEP_RN', 'DIPEP_RP', 'DIPEP_RQ', 'DIPEP_RR', 'DIPEP_RS', 'DIPEP_RT',
'DIPEP_RV', 'DIPEP_RW', 'DIPEP_RY', 'DIPEP_SA', 'DIPEP_SC', 'DIPEP_SD',
'DIPEP_SE', 'DIPEP_SF', 'DIPEP_SG', 'DIPEP_SH', 'DIPEP_SI', 'DIPEP_SK',
'DIPEP_SL', 'DIPEP_SM', 'DIPEP_SN', 'DIPEP_SP', 'DIPEP_SQ', 'DIPEP_SR',
'DIPEP_SS', 'DIPEP_ST', 'DIPEP_SV', 'DIPEP_SW', 'DIPEP_SY', 'DIPEP_TA',
'DIPEP_TC', 'DIPEP_TD', 'DIPEP_TE', 'DIPEP_TF', 'DIPEP_TG', 'DIPEP_TH',
'DIPEP_TI', 'DIPEP_TK', 'DIPEP_TL', 'DIPEP_TM', 'DIPEP_TN', 'DIPEP_TP',
'DIPEP_TQ', 'DIPEP_TR', 'DIPEP_TS', 'DIPEP_TT', 'DIPEP_TV', 'DIPEP_TW',
'DIPEP_TY', 'DIPEP_VA', 'DIPEP_VC', 'DIPEP_VD', 'DIPEP_VE', 'DIPEP_VF',
'DIPEP_VG', 'DIPEP_VH', 'DIPEP_VI', 'DIPEP_VK', 'DIPEP_VL', 'DIPEP_VM',
'DIPEP_VN', 'DIPEP_VP', 'DIPEP_VQ', 'DIPEP_VR', 'DIPEP_VS', 'DIPEP_VT',
'DIPEP_VV', 'DIPEP_VW', 'DIPEP_VY', 'DIPEP_WA', 'DIPEP_WC', 'DIPEP_WD',
'DIPEP_WE', 'DIPEP_WF', 'DIPEP_WG', 'DIPEP_WH', 'DIPEP_WI', 'DIPEP_WK',
'DIPEP_WL', 'DIPEP_WM', 'DIPEP_WN', 'DIPEP_WP', 'DIPEP_WQ', 'DIPEP_WR',
'DIPEP_WS', 'DIPEP_WT', 'DIPEP_WV', 'DIPEP_WW', 'DIPEP_WY', 'DIPEP_YA',
'DIPEP_YC', 'DIPEP_YD', 'DIPEP_YE', 'DIPEP_YF', 'DIPEP_YG', 'DIPEP_YH',
'DIPEP_YI', 'DIPEP_YK', 'DIPEP_YL', 'DIPEP_YM', 'DIPEP_YN', 'DIPEP_YP',
'DIPEP_YQ', 'DIPEP_YR', 'DIPEP_YS', 'DIPEP_YT', 'DIPEP_YV', 'DIPEP_YW',
'DIPEP_YY', 'P_LOOP', 'KINASE2', 'GLPL', 'MHD', 'RNBS_A', 'RNBS_B', 'RNBS_C',
'RNBS_D', 'HRD', 'DFG', 'LRR1', 'LRR2', 'NUM_NLR_MOTIFS',
'num_motifs_detectados', 'dist_ploop_kinase2', 'dist_kinase2_glpl',
'dist_glpl_mhd', 'dist_ploop_kinase2_norm', 'dist_kinase2_glpl_norm',
'dist_glpl_mhd_norm', 'has_ploop', 'has_kinase2', 'has_glpl', 'has_mhd']

 Clusters ausentes:
131

Dataset original pós CD-HIT: (9432, 448)
```

```
--- Executando Balanceamento de Classes ---
Contagem inicial -> Positivos (R-genes): 2336 | Negativos (Background): 7096
Subamostragem aplicada com sucesso nos Negativos.
Contagem final   -> Positivos: 2336 | Negativos: 2336
Dataset Final Pronto: (4672, 448)


Finalizado em: 15:35:56

Tempo decorrido: 0:00:03.743181


Fim tratamento dos dados…
```

[5]:
```python
'''
k-Fold-Cross-Validation

'''

def validacao_cruzada(nome, model, X_train, y_train, groups_train):

    cv = StratifiedGroupKFold(
        n_splits=5, #10
        shuffle=True,
        random_state=42
    )
    mcc_scorer = make_scorer(matthews_corrcoef)
    scoring_dict = {
        'recall': 'recall',
        'roc_auc': 'roc_auc',
        'mcc': mcc_scorer,
        'accuracy': 'accuracy',
        'precision': 'precision',
        'f1': 'f1'
    }
    scores = cross_validate(
        model,
        X_train,
        y_train,
        groups=groups_train,
        cv=cv,
        scoring=scoring_dict,
        n_jobs=-1
    )
    print(f"Finalizada a validação cruzada para: {nome}")
    return scores
```

[6]:
```
'''
Criação dos modelos/ Treinamento
'''
# para melhor separar os conjuntos de treino e teste considerando os␣
 ↪agrupamentos feitos no CD-HIT para sequências muito parecidas
def split_por_homologia(df):
    print(df.head(30))
    groups = df["cluster"]
    splitter = GroupShuffleSplit(
        n_splits=1,
        test_size=0.2,
        random_state=42
    )
    # cv  = StratifiedGroupKFold(
    #     n_splits = 10,
    #     shuffle=True,
    #     random_state = 42
    # )

    train_idx, test_idx = next(
        splitter.split(df, groups=groups)
        # cv.split(df, df["target"], groups)
    )
    train_df = df.iloc[train_idx]
    test_df = df.iloc[test_idx]
    return train_df, test_df

def calcular_e_salvar_shap(nome, model, X_test, feature_names, output_dir):
    """
    Calcula os SHAP values para modelos de árvore e salva os gráficos␣
 ↪explicativos.
    """
    print(f"\n[SHAP] Inicializando análise de interpretabilidade para: {nome}...
 ↪")

    # 1. Inicializa o explicador otimizado para árvores
    explainer = shap.TreeExplainer(model)

    # 2. Calcula os SHAP values para o conjunto de teste
    shap_values = explainer.shap_values(X_test)

    # 3. Tratamento de formato (Scikit-Learn Random Forest vs XGBoost)
    # O Random Forest do sklearn retorna uma lista contendo [valores_classe_0,␣
 ↪valores_classe_1]
    # O XGBoost retorna diretamente a matriz de SHAP values para a classe alvo
    if isinstance(shap_values, list):
```

```
        # Selecionamos o índice 1 (classe positiva: proteínas de resistência/
↪alvos)
        shap_values_pos = shap_values[1]
    elif isinstance(shap_values, np.ndarray) and len(shap_values.shape) == 3:
        # Tratamento para versões específicas do SHAP que geram arrays 3D
        shap_values_pos = shap_values[:, :, 1]
    else:
        shap_values_pos = shap_values

    # 4. Gráfico 1: Summary Plot (Beeswarm)
    # Mostra o impacto biológico de cada feature (ex: se alta frequência␣
↪aumenta ou diminui a predição)
    plt.figure(figsize=(12, 8))
    shap.summary_plot(
        shap_values_pos,
        X_test,
        feature_names=feature_names,
        max_display=20,  # Foco nas top 20 características
        show=False
    )
    # plt.title(f"SHAP Summary Plot (Top 20) - {nome}", fontsize=14, pad=20)
    plt.tight_layout()
    plt.savefig(f"{output_dir}/{nome}_shap_summary.png", bbox_inches="tight",␣
↪dpi=300)
    plt.close()

    # 5. Gráfico 2: Bar Plot (Importância Global)
    # Mostra a magnitude média do impacto de cada feature de forma absoluta
    plt.figure(figsize=(12, 8))
    shap.summary_plot(
        shap_values_pos,
        X_test,
        feature_names=feature_names,
        plot_type="bar",
        max_display=20,
        show=False
    )
    # plt.title(f"SHAP Global Feature Importance - {nome}", fontsize=14, pad=20)
    plt.tight_layout()
    plt.savefig(f"{output_dir}/{nome}_shap_bar.png", bbox_inches="tight",␣
↪dpi=300)
    plt.close()

    print(f"[SHAP] Gráficos salvos com sucesso em '{output_dir}/'!")
```

```
def calcular_shap_svm(nome, model, feature_names,  X_train, X_test, y_test,␣
↪output_dir):
    # ======================================================
    # IMPORTÂNCIA POR PERMUTAÇÃO PARA O SVM (Novo)
    # ======================================================
    print("\n[SVM] Calculando a importância por permutação dos atributos...")

    # Calcula a permutação no X_test (pode usar X_train se quiser focar no␣
↪ajuste ao treino)
    result_svm = permutation_importance(model, X_test, y_test, n_repeats=5,␣
↪random_state=42, n_jobs=-1)

    # Monta o DataFrame com os resultados
    importancia_svm = pd.DataFrame({
        'feature': columns,
        'importance_mean': result_svm.importances_mean
    }).sort_values('importance_mean', ascending=False)

    print("\nTop 20 features mais importantes para o SVM:")
    print(importancia_svm.head(20))

    # Gera o gráfico de barras igual ao do XGBoost
    top_svm = importancia_svm.head(20)
    plt.figure(figsize=(10,8))
    plt.barh(top_svm["feature"][::-1], top_svm["importance_mean"][::-1],␣
↪color='seagreen')
    plt.xlabel("Queda no Desempenho (Média)")
    plt.ylabel("Feature / Atributo")
    # plt.title("Top 20 Features - SVM Permutation Importance")
    plt.tight_layout()
    plt.savefig(f"{output_dir}/svm_importancia_permutacao.png",␣
↪bbox_inches="tight", dpi=300)
    plt.close()

    idx_back = np.random.choice(
        len(X_train),
        min(100,len(X_train)),
        replace = False
    )
    back = X_train[idx_back]

    idx_test = np.random.choice(
        len(X_test),
        min(200,len(X_test)),
        replace = False
    )
    x_test_sample = X_test[idx_test]
```

```
    print('\nCriando explainer shap...\n')
    explainer = shap.Explainer(model.predict, back)
    shap_values = explainer(x_test_sample, max_evals=1000)
    print("\nShap SHAP:",shap_values.values.shape)

    plt.figure(figsize=(12, 8))
    shap.summary_plot(
        shap_values.values,
        x_test_sample,
        feature_names=feature_names,
      #  plot_type="bar",
        max_display=20,
        show=False
    )
    # plt.title(f"SHAP Global Feature Importance - {nome}", fontsize=14, pad=20)
    plt.tight_layout()
    plt.savefig(f"{output_dir}/{nome}_shap_bar_summary.png",␣
 ↪bbox_inches="tight", dpi=300)
    plt.close()

    print(f"[SVM] Gráfico salvo com sucesso em '{output_dir}/
 ↪svm_importancia_permutacao.png'!")

# ============================================================
# TREINAMENTO - treina os modelos
# ============================================================

def avaliar_modelo(nome, model, X_train, y_train, X_test, y_test):
    print("\n============================================================")
    print(nome)
    print("============================================================")
    model.fit(X_train, y_train)

    probs = model.predict_proba(X_test)[:,1]
    preds = (probs >= 0.4).astype(int)
    print(classification_report(y_test, preds))
    roc = roc_auc_score(y_test, probs)
    pr = average_precision_score(y_test, probs)
    mcc = matthews_corrcoef(y_test, preds)
    f1 = f1_score(y_test, preds)
    precision = precision_score(y_test, preds)
    recall = recall_score(y_test, preds)
    bal_acc = balanced_accuracy_score(y_test, preds)
    acc = accuracy_score(y_test, preds)

    print("ROC-AUC          :", roc)
```

```
    print("PR-AUC           :", pr)
    print("MCC              :", mcc)
    print("F1               :", f1)
    print("Precision        :", precision)
    print("Recall           :", recall)
    print("Balanced Accuracy:", bal_acc)
    print("Standard Accuracy:", acc)

    # ========================================================
    # MATRIZ DE CONFUSÃO
    # ========================================================
    cm = confusion_matrix(y_test, preds)
    plt.figure(figsize=(5,5))
    disp = ConfusionMatrixDisplay(
        confusion_matrix=cm,
        display_labels=["Negativo", "Positivo"]
    )
    disp.plot(cmap="Blues")
    # plt.title(f"Matriz de Confusão - {nome}")
    plt.savefig(
        f"{OUTPUT_DIR}/{nome}_matriz_confusao.png",
        bbox_inches="tight"
    )
    plt.close()
    # ========================================================
    # CURVA ROC
    # ========================================================
    fpr, tpr, _ = roc_curve(y_test, probs)
    roc_auc = auc(fpr, tpr)
    plt.figure(figsize=(6,6))
    plt.plot(
        fpr,
        tpr,
        label=f"AUC = {roc_auc:.4f}"
    )
    plt.plot([0,1], [0,1], linestyle="--")
    plt.xlabel("False Positive Rate")
    plt.ylabel("True Positive Rate")
    # plt.title(f"Curva ROC - {nome}")
    plt.legend(loc="lower right")
    plt.savefig(
        f"{OUTPUT_DIR}/{nome}_roc.png",
        bbox_inches="tight"
    )
    plt.close()

    return {
```

```
        "Modelo": nome,
        "ROC_AUC": roc,
        "F1": f1,
        "Precision": precision,
        "Recall": recall,
        "Balanced_Accuracy": bal_acc,
        "Accuracy": acc,
        "MCC":mcc
    }, model


# ========================================================
# Separação Treino/Teste 80/20
# SPLIT POR HOMOLOGIA
# ========================================================
inicio_treinamento = datetime.now()
print("\n\n\nInciando Treinamento dos modelos: ", inicio_treinamento.
 ↪strftime("%H:%M:%S"))

train_df, test_df = split_por_homologia(df)
X_train = train_df[columns]
y_train = train_df["target"]
X_test = test_df[columns]
y_test = test_df["target"]
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# ========================================================
# MODELOS
# ========================================================

rf = RandomForestClassifier(
    n_estimators=1000,
    # max_depth = 12,
    max_depth = None,
    min_samples_split=5,
    max_features=0.2,
    random_state=42,
    class_weight="balanced",
    n_jobs = 1
)

xgb = XGBClassifier(
    n_estimators=1200,
    max_depth=7,
    learning_rate=0.05,
```

```
    # early_stopping_rounds=50,
    subsample=0.8,
    colsample_bytree=0.8,
    eval_metric="logloss",
    random_state=42,
    min_child_weight=3,
    reg_alpha=0.5,
    reg_lambda=2,
    scale_pos_weight=1.0
)


svm = SVC(
    probability=True,
    kernel="rbf",
    class_weight="balanced",
    random_state=42,
    C=2,
    gamma='scale'
)
scores = []
groups_train = train_df["cluster"]

# msc_scorer = make_scorer(matthews_corrcoef)

scoring = {
    'mcc': make_scorer(matthews_corrcoef),
    'recall':  make_scorer(recall_score),
    'precision':  make_scorer(precision_score),
    'f1':  make_scorer(f1_score),
    'roc_auc': 'roc_auc',
}
```

```
Inciando Treinamento dos modelos:  15:35:56
    protein_length     AAC_A     AAC_C     AAC_D     AAC_E     AAC_F  \
0              231  0.090909  0.012987  0.069264  0.077922  0.051948
1              638  0.061129  0.012539  0.047022  0.053292  0.040752
2             1249  0.032826  0.013611  0.078463  0.082466  0.044836
3              201  0.039801  0.004975  0.034826  0.044776  0.019900
4              670  0.049254  0.016418  0.052239  0.062687  0.046269
5              674  0.060831  0.011869  0.051929  0.048961  0.044510
6              888  0.055180  0.018018  0.061937  0.072072  0.045045
7              662  0.069486  0.012085  0.052870  0.067976  0.051360
8              385  0.023377  0.023377  0.044156  0.038961  0.054545
9              504  0.103175  0.019841  0.045635  0.071429  0.029762
```

10	704	0.049716	0.007102	0.080966	0.117898	0.041193
11	535	0.095327	0.009346	0.020561	0.026168	0.082243
12	677	0.051699	0.023634	0.053176	0.062038	0.051699
13	721	0.058252	0.015257	0.052705	0.055479	0.054092
14	450	0.040000	0.028889	0.035556	0.082222	0.024444
15	294	0.078231	0.017007	0.040816	0.068027	0.027211
16	1251	0.035971	0.031974	0.053557	0.067946	0.041567
17	283	0.045936	0.035336	0.060071	0.063604	0.053004
18	2293	0.037069	0.023550	0.056258	0.067161	0.038814
19	813	0.027060	0.027060	0.027060	0.045510	0.049200
20	1194	0.040201	0.018425	0.069514	0.069514	0.049414
21	1028	0.050584	0.024319	0.050584	0.040856	0.055447
22	651	0.046083	0.058372	0.064516	0.041475	0.029186
23	808	0.027228	0.023515	0.043317	0.044554	0.045792
24	906	0.030905	0.016556	0.046358	0.051876	0.059603
25	128	0.054688	0.007812	0.046875	0.203125	0.031250
26	1222	0.108838	0.010638	0.046645	0.084288	0.034370
27	261	0.026820	0.019157	0.038314	0.088123	0.038314
28	209	0.009569	0.019139	0.057416	0.038278	0.066986
29	227	0.039648	0.044053	0.101322	0.079295	0.035242

	AAC_G	AAC_H	AAC_I	AAC_K	…	dist_ploop_kinase2_norm	\
0	0.077922	0.038961	0.043290	0.082251	…	-1.000000	
1	0.062696	0.028213	0.062696	0.073668	…	-1.000000	
2	0.040032	0.029624	0.064051	0.077662	…	0.072058	
3	0.054726	0.024876	0.074627	0.084577	…	-1.000000	
4	0.073134	0.016418	0.079104	0.046269	…	-1.000000	
5	0.084570	0.028190	0.065282	0.040059	…	-1.000000	
6	0.047297	0.023649	0.061937	0.085586	…	0.099099	
7	0.066465	0.016616	0.058912	0.033233	…	-1.000000	
8	0.062338	0.015584	0.083117	0.059740	…	-1.000000	
9	0.081349	0.005952	0.075397	0.073413	…	-1.000000	
10	0.048295	0.009943	0.065341	0.117898	…	-1.000000	
11	0.106542	0.014953	0.065421	0.039252	…	-1.000000	
12	0.067947	0.022157	0.066470	0.042836	…	-1.000000	
13	0.084605	0.020804	0.054092	0.047157	…	-1.000000	
14	0.048889	0.006667	0.075556	0.091111	…	-1.000000	
15	0.047619	0.020408	0.051020	0.074830	…	-1.000000	
16	0.055156	0.028777	0.067946	0.064748	…	-1.000000	
17	0.053004	0.010601	0.067138	0.045936	…	-1.000000	
18	0.058003	0.028347	0.068905	0.073266	…	-1.000000	
19	0.077491	0.019680	0.059041	0.028290	…	-1.000000	
20	0.057789	0.019263	0.064489	0.067839	…	-1.000000	
21	0.082685	0.015564	0.051556	0.055447	…	-1.000000	
22	0.069124	0.024578	0.056836	0.061444	…	-1.000000	
23	0.063119	0.017327	0.075495	0.048267	…	-1.000000	
24	0.058499	0.011038	0.076159	0.044150	…	-1.000000	
25	0.039062	0.015625	0.070312	0.101562	…	-1.000000	

```
26  0.070376  0.009820  0.056465  0.049100  …               -1.000000
27  0.022989  0.038314  0.076628  0.084291  …               -1.000000
28  0.028708  0.033493  0.057416  0.057416  …               -1.000000
29  0.048458  0.017621  0.061674  0.057269  …               -1.000000
```

```
    dist_kinase2_glpl_norm  dist_glpl_mhd_norm  has_ploop  has_kinase2  \
0                -1.000000                   1          0            0
1                -1.000000                   0          0            1
2                -0.304243                   1          1            1
3                -1.000000                   0          0            0
4                 0.038806                   1          0            1
5                 0.203264                   0          0            1
6                 0.117117                   1          1            1
7                -1.000000                   0          0            1
8                -1.000000                   0          0            0
9                -1.000000                   1          0            0
10               -1.000000                   0          0            1
11               -1.000000                   0          0            1
12               -1.000000                   0          0            1
13               -1.000000                   1          0            1
14               -1.000000                   0          0            0
15                0.455782                   0          0            1
16               -1.000000                   1          1            0
17               -1.000000                   0          0            1
18               -0.047100                   1          0            1
19               -0.653137                   0          0            1
20               -1.000000                   1          0            0
21                0.256809                   1          0            1
22               -0.162826                   0          0            1
23               -1.000000                   0          0            0
24               -0.005519                   1          0            1
25               -1.000000                   0          0            0
26               -0.180033                   1          0            1
27               -1.000000                   0          0            0
28               -1.000000                   0          0            0
29               -1.000000                   0          0            0
```

```
    has_glpl  has_mhd  target              id   cluster
0          0        1       1  NP_001234459.3  POS_2155
1          0        1       1  NP_001306257.2  POS_1204
2          1        4       1  NP_001318035.2   POS_183
3          1        1       1      YEV57985.1  POS_2239
4          1        3       1  NP_001307923.1  POS_1146
5          1        2       1  NP_001307848.1  POS_1138
6          1        4       1  NP_001234202.1   POS_751
7          0        1       1  NP_001307817.1  POS_1161
8          0        0       1  NP_001355132.1  POS_1803
9          0        1       1  NP_001333606.1  POS_1483
```

```
10          0         1         1  NP_001308492.1  POS_1078
11          0         1         1  NP_001307892.1  POS_1402
12          0         1         1  NP_001307546.1  POS_1135
13          0         2         1  NP_001307224.1  POS_1040
14          0         0         1  NP_001234240.1  POS_1623
15          1         2         1    CAQ5401959.1  POS_2012
16          1         3         1    CAQ5402156.1   POS_179
17          0         1         1    CAQ5401373.1  POS_2035
18          1         3         1    CAQ5402324.1    POS_12
19          1         2         1    CAQ5401369.1   POS_892
20          1         2         1    CAQ5401367.1   POS_239
21          1         3         1    CAQ5401354.1   POS_424
22          1         2         1    CAQ5402419.1  POS_1183
23          0         0         1    CAQ5402454.1   POS_899
24          1         3         1    CAQ5401193.1   POS_698
25          1         1         1    CAQ5401117.1  POS_2363
26          1         3         1    CAQ5402682.1   POS_210
27          1         1         1    CAQ5400846.1  POS_2083
28          0         0         1    CAQ5400845.1  POS_2216
29          0         0         1    CAQ5400813.1  POS_2162

[30 rows x 448 columns]
```

[7]:
```
inicio_otimizacao = datetime.now()
print("\n\n\nInciando Otimização dos modelos: ", inicio_otimizacao.strftime("%H:
  ↪%M:%S"))

cv_t = GroupKFold(n_splits=5)

param_dist_xgb = {
    'n_estimators':[500,800,1200,1500],
    'max_depth':[3,4,5,6,7,10],
    'learning_rate':[0.01,0.03,0.05,0.1],
    'subsample':[0.7,0.8,0.9],
    'colsample_bytree':[0.7,0.8,0.9],
    'min_child_weight':[1,3,5],
    'reg_alpha':[0,0.1,0.5,0.7],
    'reg_lambda':[1,2,5]
}
# Otimização XGB
print('\n\nOtimizando XGBoost...')
search_xgb = RandomizedSearchCV(
    estimator=xgb,
    param_distributions=param_dist_xgb,
    n_iter=50,
    scoring=scoring,
    refit='f1',
```

```
    cv=cv_t,
    random_state=42,
    verbose=2,
    n_jobs=-1
)

search_xgb.fit(
    X_train,
    y_train,
    groups=groups_train
)
print("\nMelhores parâmetros para o XGB:")
print(search_xgb.best_params_)

print("\nMelhor Recall:")
print(search_xgb.best_score_)

best_params = search_xgb.best_params_
xgb = search_xgb.best_estimator_
```

```
Inciando Otimização dos modelos:  15:35:56


Otimizando XGBoost…
Fitting 5 folds for each of 50 candidates, totalling 250 fits

Melhores parâmetros para o XGB:
{'subsample': 0.9, 'reg_lambda': 1, 'reg_alpha': 0, 'n_estimators': 500,
'min_child_weight': 1, 'max_depth': 6, 'learning_rate': 0.05,
'colsample_bytree': 0.8}

Melhor Recall:
0.8722713447817145
```

[8]:
```
# otimização RF

param_dist_rf = {
    'n_estimators':[500,1000,1200,1500],
    'max_depth':[8,12,16, None],
    'min_samples_split':[2,4,5,8],
    'min_samples_leaf':[1,2,4],
    'max_features':['sqrt','log2',0,3],
    'class_weight':['balanced','balanced_subsample']
}
```

```
print('\n\nOtimizando RF...')
search_rf = RandomizedSearchCV(
    estimator=rf,
    param_distributions=param_dist_rf,
    n_iter=40,
    scoring=scoring,
    refit='f1',
    cv=cv_t,
    random_state=42,
    verbose=2,
    n_jobs=-1
)

search_rf.fit(
    X_train,
    y_train,
    groups=groups_train
)
print("\nMelhores parâmetros para o RF:")
print(search_rf.best_params_)

print("\nMelhor Recall:")
print(search_rf.best_score_)

rf = search_rf.best_estimator_

rf.set_params(n_jobs=-1)
```

```
Otimizando RF…
Fitting 5 folds for each of 40 candidates, totalling 200 fits

Melhores parâmetros para o RF:
{'n_estimators': 1000, 'min_samples_split': 4, 'min_samples_leaf': 1,
'max_features': 'sqrt', 'max_depth': 16, 'class_weight': 'balanced_subsample'}

Melhor Recall:
0.8543508653517646
```

[8]:
```
RandomForestClassifier(class_weight='balanced_subsample', max_depth=16,
                       min_samples_split=4, n_estimators=1000, n_jobs=-1,
                       random_state=42)
```

[9]:
```
print('\n\nOtimizando SVM...')
param_dist_svm = {
    'C':[0.5,1,2,5,10,20,50],
```

```
    'gamma':[
        'scale',
        0.01,
        0.005,
        0.001,
        0.0005
    ],
    'kernel':['rbf']
}
search_svm = RandomizedSearchCV(
    estimator=svm,
    param_distributions=param_dist_svm,
    n_iter=25,
    scoring=scoring,
    refit='f1',
    cv=cv_t,
    random_state=42,
    verbose=2,
    n_jobs=-1
)

search_svm.fit(
    X_train,
    y_train,
    groups=groups_train
)
print("\nMelhores parâmetros para o SVM:")
print(search_svm.best_params_)

print("\nMelhor Recall:")
print(search_svm.best_score_)

svm = search_svm.best_estimator_

fim = datetime.now()
print("\n\nFinalizado otimização dos modelos em:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio_otimizacao)
# Fim otimização
```

```
Otimizando SVM…
Fitting 5 folds for each of 25 candidates, totalling 125 fits

Melhores parâmetros para o SVM:
{'kernel': 'rbf', 'gamma': 'scale', 'C': 5}

Melhor Recall:
```

```
0.8786931391007459


Finalizado otimização dos modelos em: 15:57:35

Tempo decorrido: 0:21:38.883016
```

[10]:
```python
# xgb = search_xgb.best_estimator_

print("\n\n Inciando validação cruzada");
scores_rf = validacao_cruzada('RF',rf,X_train, y_train, groups_train)
scores_xgb = validacao_cruzada('XGB',xgb, X_train, y_train, groups_train)
scores_svm = validacao_cruzada('SVM',svm, X_train, y_train, groups_train)

print("Cross-Validation:RF")
for k,v in scores_rf.items():
    if "test" in k:
        print(f"{k}: {v.mean():.4f} ± {v.std():.4f}")

print("Cross-Validation XGB")
for k,v in scores_xgb.items():
    if "test" in k:
        print(f"{k}: {v.mean():.4f} ± {v.std():.4f}")

print("Cross-Validation SVM")
for k,v in scores_svm.items():
    if "test" in k:
        print(f"{k}: {v.mean():.4f} ± {v.std():.4f}")


resultados_cv = pd.DataFrame({
    'Modelo': ['RF','XGB','SVM'],
    'Accuracy': [
        scores_rf['test_accuracy'].mean(),
        scores_xgb['test_accuracy'].mean(),
        scores_svm['test_accuracy'].mean(),
    ],
    'Recall': [
        scores_rf['test_recall'].mean(),
        scores_xgb['test_recall'].mean(),
        scores_svm['test_recall'].mean(),
    ],
    'Precision': [
        scores_rf['test_precision'].mean(),
        scores_xgb['test_precision'].mean(),
        scores_svm['test_precision'].mean(),
    ],
```

```
    'F1': [
        scores_rf['test_f1'].mean(),
        scores_xgb['test_f1'].mean(),
        scores_svm['test_f1'].mean(),
    ],
    'ROC_AUC': [
        scores_rf['test_roc_auc'].mean(),
        scores_xgb['test_roc_auc'].mean(),
        scores_svm['test_roc_auc'].mean(),
    ],
    'MCC': [
        scores_rf['test_mcc'].mean(),
        scores_xgb['test_mcc'].mean(),
        scores_svm['test_mcc'].mean(),
    ]
})
print(resultados_cv)
resultados_cv.to_csv(
        f"{OUTPUT_DIR}/metricas_cv.csv",
        index=False
    )
dados = []

for score in scores_rf['test_roc_auc']:
    dados.append(['RF',score])
for score in scores_xgb['test_roc_auc']:
    dados.append(['XGB',score])
for score in scores_svm['test_roc_auc']:
    dados.append(['SVM',score])

grafico = pd.DataFrame(dados, columns=['Modelo','ROC_AUC'])
grafico.boxplot(by='Modelo')
plt.ylabel("ROC-AUC")
# plt.title("10-Fold Cross Validation")
plt.suptitle("")
plt.tight_layout()
# plt.show()
plt.savefig(
    f"{OUTPUT_DIR}/kfold_cross_validation_roc.png",
    bbox_inches="tight"
)
plt.close()
```

```
 Inciando validação cruzada
Finalizada a validação cruzada para: RF
Finalizada a validação cruzada para: XGB
```

```
Finalizada a validação cruzada para: SVM
Cross-Validation:RF
test_recall: 0.7751 ± 0.0159
test_roc_auc: 0.9301 ± 0.0139
test_mcc: 0.7473 ± 0.0240
test_accuracy: 0.8673 ± 0.0122
test_precision: 0.9500 ± 0.0120
test_f1: 0.8537 ± 0.0141
Cross-Validation XGB
test_recall: 0.8174 ± 0.0053
test_roc_auc: 0.9385 ± 0.0117
test_mcc: 0.7627 ± 0.0135
test_accuracy: 0.8785 ± 0.0064
test_precision: 0.9312 ± 0.0103
test_f1: 0.8706 ± 0.0066
Cross-Validation SVM
test_recall: 0.8308 ± 0.0180
test_roc_auc: 0.9410 ± 0.0096
test_mcc: 0.7721 ± 0.0229
test_accuracy: 0.8839 ± 0.0118
test_precision: 0.9293 ± 0.0084
test_f1: 0.8772 ± 0.0133
  Modelo  Accuracy    Recall  Precision        F1   ROC_AUC       MCC
0     RF  0.867271  0.775143   0.950050  0.853706  0.930092  0.747299
1    XGB  0.878510  0.817448   0.931168  0.870591  0.938534  0.762748
2    SVM  0.883863  0.830822   0.929259  0.877240  0.940958  0.772105
```

[11]:
```
################################################
# Treinamento
inicio = datetime.now()
print("\n\n\nInciando treinamento RF: ", inicio.strftime("%H:%M:%S"))
results = []
r1, rf_model = avaliar_modelo(
    "Random Forest",
    rf,
    X_train,
    y_train,
    X_test,
    y_test
)
results.append(r1)
fim = datetime.now()
print("\n\nFinalizado treinamento RF em:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)

calcular_e_salvar_shap("Random Forest", rf_model, X_test, columns,OUTPUT_DIR )
```

```
inicio = datetime.now()
print("\n\n\nInciando treinamento XGBoost: ", inicio.strftime("%H:%M:%S"))
r2, xgb_model = avaliar_modelo(
    "XGBoost",
    xgb,
    X_train,
    y_train,
    X_test,
    y_test
)
results.append(r2)
fim = datetime.now()
print("\n\nFinalizado em XGBoost:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)

calcular_e_salvar_shap("XGBoost", xgb_model, X_test, columns,OUTPUT_DIR )

inicio = datetime.now()
print("\n\n\nInciando treinamento SVM: ", inicio.strftime("%H:%M:%S"))
r3, svm_model = avaliar_modelo(
    "SVM",
    svm,
    X_train,
    y_train,
    X_test,
    y_test
)
results.append(r3)
fim = datetime.now()
print("\n\nFinalizado em SVM:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)

calcular_shap_svm("SVM", svm_model,columns, X_train, X_test, y_test, OUTPUT_DIR)

fim = datetime.now()
print("\n\nFinalizado fase de treinamento em:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio_treinamento)

print("\nFim criação/treino dos modelos\n")
```

```
Inciando treinamento RF:  15:58:11

============================================================
Random Forest
============================================================
```

```
              precision    recall  f1-score   support

           0       0.85      0.88      0.86       467
           1       0.87      0.85      0.86       468

    accuracy                           0.86       935
   macro avg       0.86      0.86      0.86       935
weighted avg       0.86      0.86      0.86       935

ROC-AUC          : 0.9424129284942989
PR-AUC           : 0.9540532270190085
MCC              : 0.7264497562431056
F1               : 0.8614718614718615
Precision        : 0.8728070175438597
Recall           : 0.8504273504273504
Balanced Accuracy: 0.8631151741430114
Standard Accuracy: 0.8631016042780749


Finalizado treinamento RF em: 15:58:14

Tempo decorrido: 0:00:02.707263

[SHAP] Inicializando análise de interpretabilidade para: Random Forest…
[SHAP] Gráficos salvos com sucesso em 'pipelineAD/'!



Inciando treinamento XGBoost:  15:59:25

============================================================
XGBoost
============================================================
              precision    recall  f1-score   support

           0       0.86      0.92      0.89       467
           1       0.91      0.85      0.88       468

    accuracy                           0.88       935
   macro avg       0.89      0.88      0.88       935
weighted avg       0.89      0.88      0.88       935

ROC-AUC          : 0.9502141327623127
PR-AUC           : 0.9601398898324739
MCC              : 0.7690211760165453
F1               : 0.8790233074361821
Precision        : 0.9145496535796767
Recall           : 0.8461538461538461
```

```
Balanced Accuracy: 0.8834623620490858
Standard Accuracy: 0.8834224598930481


Finalizado em XGBoost: 15:59:31

Tempo decorrido: 0:00:06.398321

[SHAP] Inicializando análise de interpretabilidade para: XGBoost…
[SHAP] Gráficos salvos com sucesso em 'pipelineAD/'!



Inciando treinamento SVM:  15:59:32

============================================================
SVM
============================================================
              precision    recall  f1-score   support

           0       0.88      0.87      0.87       467
           1       0.87      0.88      0.87       468

    accuracy                           0.87       935
   macro avg       0.87      0.87      0.87       935
weighted avg       0.87      0.87      0.87       935

ROC-AUC          : 0.9454419004740203
PR-AUC           : 0.9550887306633454
MCC              : 0.7454678566067147
F1               : 0.873269435569755
Precision        : 0.8704883227176221
Recall           : 0.8760683760683761
Balanced Accuracy: 0.8727236955288347
Standard Accuracy: 0.8727272727272727


Finalizado em SVM: 15:59:36

Tempo decorrido: 0:00:04.485418

[SVM] Calculando a importância por permutação dos atributos…

Top 20 features mais importantes para o SVM:
      feature  importance_mean
250  DIPEP_NL         0.006845
42   DIPEP_CC         0.005134
77   DIPEP_DT         0.004278
```

```
407  DIPEP_YH         0.004064
121  DIPEP_GA         0.003422
144  DIPEP_HE         0.003422
102  DIPEP_FC         0.003422
401  DIPEP_YA         0.003422
150  DIPEP_HL         0.003422
241  DIPEP_NA         0.003209
260  DIPEP_NY         0.003209
68   DIPEP_DI         0.003209
303  DIPEP_RD         0.002995
332  DIPEP_SN         0.002781
289  DIPEP_QK         0.002781
299  DIPEP_QW         0.002781
43   DIPEP_CD         0.002781
428    RNBS_D         0.002781
300  DIPEP_QY         0.002567
63   DIPEP_DD         0.002353

Criando explainer shap…


PermutationExplainer explainer:   5%|        | 10/200 [04:41<1:40:30, 31.74s/it]

[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0.5, reg_lambda=5,
subsample=0.9; total time=  20.9s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=5,
min_child_weight=5, n_estimators=1500, reg_alpha=0.7, reg_lambda=2,
subsample=0.7; total time=  52.7s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=6,
min_child_weight=1, n_estimators=1200, reg_alpha=0.1, reg_lambda=2,
subsample=0.9; total time= 2.9min
[CV] END colsample_bytree=0.8, learning_rate=0.01, max_depth=5,
min_child_weight=1, n_estimators=1500, reg_alpha=0.5, reg_lambda=2,
subsample=0.9; total time= 2.3min
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=6,
min_child_weight=5, n_estimators=1200, reg_alpha=0.5, reg_lambda=1,
subsample=0.8; total time=  49.1s
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=10,
min_child_weight=5, n_estimators=500, reg_alpha=0.1, reg_lambda=1,
subsample=0.9; total time= 1.0min
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0.5, reg_lambda=2,
subsample=0.9; total time=  25.5s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=5,
min_child_weight=5, n_estimators=800, reg_alpha=0.5, reg_lambda=5,
subsample=0.7; total time=  49.8s
[CV] END colsample_bytree=0.7, learning_rate=0.1, max_depth=5,
min_child_weight=1, n_estimators=500, reg_alpha=0.5, reg_lambda=5,
```

```
subsample=0.9; total time=  42.4s
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=4,
min_child_weight=5, n_estimators=1200, reg_alpha=0, reg_lambda=5, subsample=0.7;
total time=  41.6s
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=3,
min_child_weight=3, n_estimators=500, reg_alpha=0.1, reg_lambda=1,
subsample=0.9; total time=  13.1s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=10,
min_child_weight=1, n_estimators=800, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time= 2.0min
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=log2,
min_samples_leaf=4, min_samples_split=2, n_estimators=500; total time=   6.8s
[CV] END class_weight=balanced, max_depth=None, max_features=3,
min_samples_leaf=4, min_samples_split=4, n_estimators=1000; total time=   6.8s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=sqrt,
min_samples_leaf=1, min_samples_split=4, n_estimators=500; total time=  15.5s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time=  29.7s
[CV] END class_weight=balanced, max_depth=12, max_features=3,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time=   6.5s
[CV] END class_weight=balanced, max_depth=12, max_features=3,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time=   6.6s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=3,
min_samples_leaf=2, min_samples_split=4, n_estimators=1000; total time=   7.9s
[CV] END class_weight=balanced, max_depth=None, max_features=0,
min_samples_leaf=4, min_samples_split=2, n_estimators=500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=8, max_features=3,
min_samples_leaf=2, min_samples_split=8, n_estimators=1000; total time=   6.7s
[CV] END class_weight=balanced, max_depth=16, max_features=3,
min_samples_leaf=4, min_samples_split=8, n_estimators=1500; total time=  10.4s
[CV] END class_weight=balanced, max_depth=None, max_features=sqrt,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time=  27.3s
[CV] END …C=10, gamma=0.0005, kernel=rbf; total time=  38.1s
[CV] END …C=5, gamma=scale, kernel=rbf; total time= 1.1min
[CV] END …C=1, gamma=0.0005, kernel=rbf; total time=  43.8s
[CV] END …C=0.5, gamma=scale, kernel=rbf; total time=  52.0s
[CV] END …C=1, gamma=scale, kernel=rbf; total time=  51.5s
[CV] END …C=50, gamma=0.005, kernel=rbf; total time= 1.3min
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=4,
min_child_weight=5, n_estimators=1500, reg_alpha=0.7, reg_lambda=2,
subsample=0.9; total time= 1.1min
[CV] END colsample_bytree=0.8, learning_rate=0.01, max_depth=5,
min_child_weight=1, n_estimators=800, reg_alpha=0.1, reg_lambda=5,
subsample=0.8; total time= 1.2min
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=6,
min_child_weight=1, n_estimators=1200, reg_alpha=0.7, reg_lambda=2,
subsample=0.7; total time= 1.2min
[CV] END colsample_bytree=0.8, learning_rate=0.03, max_depth=6,
```

```
min_child_weight=3, n_estimators=500, reg_alpha=0.5, reg_lambda=2,
subsample=0.9; total time=  57.2s
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=6,
min_child_weight=3, n_estimators=1500, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time= 1.4min
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=4,
min_child_weight=5, n_estimators=800, reg_alpha=0.1, reg_lambda=2,
subsample=0.8; total time=  39.4s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=6,
min_child_weight=5, n_estimators=1200, reg_alpha=0.5, reg_lambda=1,
subsample=0.8; total time=  49.4s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=5,
min_child_weight=5, n_estimators=1200, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time= 1.1min
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0.5, reg_lambda=2,
subsample=0.9; total time=  25.9s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=5,
min_child_weight=5, n_estimators=800, reg_alpha=0.5, reg_lambda=5,
subsample=0.7; total time=  49.3s
[CV] END colsample_bytree=0.7, learning_rate=0.1, max_depth=5,
min_child_weight=1, n_estimators=500, reg_alpha=0.5, reg_lambda=5,
subsample=0.9; total time=  42.3s
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=4,
min_child_weight=5, n_estimators=1200, reg_alpha=0, reg_lambda=5, subsample=0.7;
total time=  40.3s
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=3,
min_child_weight=3, n_estimators=500, reg_alpha=0.1, reg_lambda=1,
subsample=0.9; total time=  13.3s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=10,
min_child_weight=1, n_estimators=800, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time= 1.9min
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=log2,
min_samples_leaf=4, min_samples_split=2, n_estimators=500; total time=   6.7s
[CV] END class_weight=balanced, max_depth=None, max_features=3,
min_samples_leaf=4, min_samples_split=4, n_estimators=1000; total time=   6.7s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=sqrt,
min_samples_leaf=1, min_samples_split=4, n_estimators=500; total time=  15.4s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=sqrt,
min_samples_leaf=1, min_samples_split=4, n_estimators=500; total time=  15.4s
[CV] END class_weight=balanced, max_depth=8, max_features=log2,
min_samples_leaf=1, min_samples_split=2, n_estimators=1000; total time=  10.6s
[CV] END class_weight=balanced, max_depth=8, max_features=log2,
min_samples_leaf=1, min_samples_split=2, n_estimators=1000; total time=  10.7s
[CV] END class_weight=balanced, max_depth=8, max_features=log2,
min_samples_leaf=2, min_samples_split=4, n_estimators=1200; total time=  12.7s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=3,
min_samples_leaf=2, min_samples_split=4, n_estimators=1000; total time=   7.7s
```

```
[CV] END class_weight=balanced, max_depth=16, max_features=3,
min_samples_leaf=4, min_samples_split=5, n_estimators=1200; total time=   8.1s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=2, min_samples_split=4, n_estimators=1200; total time=  32.9s
[CV] END …C=10, gamma=0.01, kernel=rbf; total time= 1.2min
[CV] END …C=5, gamma=0.0005, kernel=rbf; total time=  24.6s
[CV] END …C=5, gamma=0.01, kernel=rbf; total time= 1.1min
[CV] END …C=5, gamma=0.005, kernel=rbf; total time= 1.1min
[CV] END …C=0.5, gamma=0.01, kernel=rbf; total time= 1.3min
[CV] END …C=10, gamma=0.001, kernel=rbf; total time=  35.3s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=4,
min_child_weight=5, n_estimators=1500, reg_alpha=0.7, reg_lambda=2,
subsample=0.9; total time= 1.1min
[CV] END colsample_bytree=0.8, learning_rate=0.01, max_depth=5,
min_child_weight=1, n_estimators=800, reg_alpha=0.1, reg_lambda=5,
subsample=0.8; total time= 1.2min
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=5,
min_child_weight=5, n_estimators=1200, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time= 1.5min
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=7,
min_child_weight=5, n_estimators=1200, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time= 1.2min
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=4,
min_child_weight=1, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.7;
total time=  39.6s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=6,
min_child_weight=5, n_estimators=1200, reg_alpha=0.7, reg_lambda=1,
subsample=0.9; total time= 1.7min
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=5,
min_child_weight=5, n_estimators=1200, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time= 1.0min
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0.5, reg_lambda=2,
subsample=0.9; total time=  26.1s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=5,
min_child_weight=5, n_estimators=800, reg_alpha=0.5, reg_lambda=5,
subsample=0.7; total time=  48.5s
[CV] END colsample_bytree=0.7, learning_rate=0.1, max_depth=5,
min_child_weight=1, n_estimators=500, reg_alpha=0.5, reg_lambda=5,
subsample=0.9; total time=  42.9s
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=4,
min_child_weight=5, n_estimators=1200, reg_alpha=0, reg_lambda=5, subsample=0.7;
total time=  42.0s
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=3,
min_child_weight=3, n_estimators=500, reg_alpha=0.1, reg_lambda=1,
subsample=0.9; total time=  13.9s
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=4,
min_child_weight=1, n_estimators=500, reg_alpha=0.7, reg_lambda=2,
```

```
subsample=0.7; total time=  28.7s
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=3,
min_child_weight=5, n_estimators=1500, reg_alpha=0.1, reg_lambda=5,
subsample=0.9; total time=  43.0s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=4,
min_child_weight=5, n_estimators=800, reg_alpha=0.7, reg_lambda=5,
subsample=0.8; total time=  35.0s
[CV] END class_weight=balanced, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=sqrt,
min_samples_leaf=1, min_samples_split=8, n_estimators=1000; total time=  33.8s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=sqrt,
min_samples_leaf=4, min_samples_split=5, n_estimators=1000; total time=  28.7s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=sqrt,
min_samples_leaf=4, min_samples_split=5, n_estimators=1000; total time=  29.2s
[CV] END class_weight=balanced, max_depth=None, max_features=sqrt,
min_samples_leaf=1, min_samples_split=5, n_estimators=1000; total time=  30.6s
[CV] END …C=2, gamma=0.001, kernel=rbf; total time=  41.9s
[CV] END …C=20, gamma=0.0005, kernel=rbf; total time=  36.0s
[CV] END …C=2, gamma=0.005, kernel=rbf; total time= 1.3min
[CV] END …C=0.5, gamma=0.0005, kernel=rbf; total time=  46.7s
[CV] END …C=2, gamma=0.01, kernel=rbf; total time= 1.4min
[CV] END …C=0.5, gamma=0.001, kernel=rbf; total time=  45.1s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=4,
min_child_weight=5, n_estimators=1500, reg_alpha=0.7, reg_lambda=2,
subsample=0.9; total time= 1.0min
[CV] END colsample_bytree=0.8, learning_rate=0.01, max_depth=5,
min_child_weight=1, n_estimators=800, reg_alpha=0.1, reg_lambda=5,
subsample=0.8; total time= 1.2min
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=6,
min_child_weight=1, n_estimators=1200, reg_alpha=0.7, reg_lambda=2,
subsample=0.7; total time= 1.2min
[CV] END colsample_bytree=0.8, learning_rate=0.03, max_depth=6,
min_child_weight=3, n_estimators=500, reg_alpha=0.5, reg_lambda=2,
subsample=0.9; total time=  55.7s
[CV] END colsample_bytree=0.8, learning_rate=0.01, max_depth=5,
min_child_weight=1, n_estimators=1500, reg_alpha=0.5, reg_lambda=2,
subsample=0.9; total time= 2.3min
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=3,
min_child_weight=1, n_estimators=500, reg_alpha=0, reg_lambda=2, subsample=0.9;
total time=  18.3s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=10,
min_child_weight=3, n_estimators=1500, reg_alpha=0.5, reg_lambda=2,
subsample=0.7; total time= 3.9min
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=3,
min_child_weight=3, n_estimators=1500, reg_alpha=0.7, reg_lambda=5,
subsample=0.9; total time=  43.9s
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=5,
```

```
min_child_weight=1, n_estimators=1500, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time=  59.8s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=1, min_samples_split=4, n_estimators=1500; total time=  49.7s
[CV] END class_weight=balanced, max_depth=12, max_features=log2,
min_samples_leaf=1, min_samples_split=2, n_estimators=500; total time=   6.7s
[CV] END class_weight=balanced, max_depth=12, max_features=log2,
min_samples_leaf=1, min_samples_split=2, n_estimators=500; total time=   6.7s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=log2,
min_samples_leaf=1, min_samples_split=8, n_estimators=1200; total time=  16.9s
[CV] END class_weight=balanced_subsample, max_depth=8, max_features=3,
min_samples_leaf=2, min_samples_split=8, n_estimators=1000; total time=   6.4s
[CV] END class_weight=balanced_subsample, max_depth=8, max_features=0,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=8, max_features=0,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=8, max_features=0,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=8, max_features=0,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=8, max_features=0,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time=   0.0s
[CV] END class_weight=balanced, max_depth=16, max_features=3,
min_samples_leaf=4, min_samples_split=8, n_estimators=1500; total time=  10.1s
[CV] END class_weight=balanced, max_depth=None, max_features=sqrt,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time=  26.4s
[CV] END …C=2, gamma=0.001, kernel=rbf; total time=  41.3s
[CV] END …C=20, gamma=0.0005, kernel=rbf; total time=  36.7s
[CV] END …C=2, gamma=0.005, kernel=rbf; total time= 1.2min
[CV] END …C=0.5, gamma=scale, kernel=rbf; total time=  50.8s
[CV] END …C=2, gamma=0.01, kernel=rbf; total time= 1.4min
[CV] END …C=0.5, gamma=0.001, kernel=rbf; total time=  44.0s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=3,
min_child_weight=5, n_estimators=800, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time=  21.1s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=5,
min_child_weight=5, n_estimators=1500, reg_alpha=0.7, reg_lambda=2,
subsample=0.7; total time=  55.4s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=6,
min_child_weight=1, n_estimators=1200, reg_alpha=0.1, reg_lambda=2,
subsample=0.9; total time= 3.1min
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=6,
min_child_weight=3, n_estimators=1500, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time= 1.5min
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=4,
min_child_weight=5, n_estimators=800, reg_alpha=0.1, reg_lambda=2,
subsample=0.8; total time=  37.2s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=6,
```

```
min_child_weight=5, n_estimators=1200, reg_alpha=0.5, reg_lambda=1,
subsample=0.8; total time=  49.2s
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=10,
min_child_weight=5, n_estimators=500, reg_alpha=0.1, reg_lambda=1,
subsample=0.9; total time= 1.1min
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0.5, reg_lambda=2,
subsample=0.9; total time=  26.6s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=5,
min_child_weight=5, n_estimators=800, reg_alpha=0.5, reg_lambda=5,
subsample=0.7; total time=  47.6s
[CV] END colsample_bytree=0.7, learning_rate=0.1, max_depth=5,
min_child_weight=1, n_estimators=500, reg_alpha=0.5, reg_lambda=5,
subsample=0.9; total time=  42.8s
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=4,
min_child_weight=5, n_estimators=1200, reg_alpha=0, reg_lambda=5, subsample=0.7;
total time=  42.8s
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=3,
min_child_weight=3, n_estimators=500, reg_alpha=0.1, reg_lambda=1,
subsample=0.9; total time=  14.3s
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=4,
min_child_weight=1, n_estimators=500, reg_alpha=0.7, reg_lambda=2,
subsample=0.7; total time=  26.8s
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=3,
min_child_weight=5, n_estimators=1500, reg_alpha=0.1, reg_lambda=5,
subsample=0.9; total time=  39.9s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=4,
min_child_weight=5, n_estimators=800, reg_alpha=0.7, reg_lambda=5,
subsample=0.8; total time=  28.2s
[CV] END class_weight=balanced, max_depth=8, max_features=log2,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time=   5.4s
[CV] END class_weight=balanced_subsample, max_depth=8, max_features=3,
min_samples_leaf=1, min_samples_split=8, n_estimators=500; total time=   3.2s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=3,
min_samples_leaf=2, min_samples_split=2, n_estimators=1000; total time=   8.0s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=8, n_estimators=1500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=8, n_estimators=1500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=8, n_estimators=1500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=8, n_estimators=1500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=8, n_estimators=1500; total time=   0.0s
[CV] END class_weight=balanced, max_depth=None, max_features=0,
min_samples_leaf=1, min_samples_split=4, n_estimators=1200; total time=   0.0s
[CV] END class_weight=balanced, max_depth=None, max_features=0,
```

```
min_samples_leaf=1, min_samples_split=4, n_estimators=1200; total time=   0.0s
[CV] END class_weight=balanced, max_depth=None, max_features=0,
min_samples_leaf=1, min_samples_split=4, n_estimators=1200; total time=   0.0s
[CV] END class_weight=balanced, max_depth=12, max_features=3,
min_samples_leaf=4, min_samples_split=2, n_estimators=1500; total time=   9.4s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=0,
min_samples_leaf=4, min_samples_split=8, n_estimators=1500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=1, min_samples_split=4, n_estimators=1000; total time=  33.3s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=3,
min_samples_leaf=4, min_samples_split=5, n_estimators=1500; total time=  11.5s
[CV] END class_weight=balanced, max_depth=8, max_features=log2,
min_samples_leaf=2, min_samples_split=4, n_estimators=1200; total time=  12.8s
[CV] END class_weight=balanced, max_depth=16, max_features=3,
min_samples_leaf=4, min_samples_split=5, n_estimators=1200; total time=   8.0s
[CV] END class_weight=balanced, max_depth=None, max_features=sqrt,
min_samples_leaf=1, min_samples_split=5, n_estimators=1000; total time=  30.6s
[CV] END …C=2, gamma=0.001, kernel=rbf; total time=  42.7s
[CV] END …C=20, gamma=0.0005, kernel=rbf; total time=  36.4s
[CV] END …C=2, gamma=0.005, kernel=rbf; total time= 1.3min
[CV] END …C=0.5, gamma=0.0005, kernel=rbf; total time=  47.9s
[CV] END …C=2, gamma=0.01, kernel=rbf; total time= 1.4min
[CV] END …C=0.5, gamma=0.001, kernel=rbf; total time=  46.1s
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=3,
min_child_weight=3, n_estimators=1200, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time=  32.5s
[CV] END colsample_bytree=0.8, learning_rate=0.01, max_depth=6,
min_child_weight=5, n_estimators=500, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time=  46.6s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=5,
min_child_weight=3, n_estimators=1200, reg_alpha=0, reg_lambda=1, subsample=0.8;
total time=  56.8s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=6,
min_child_weight=1, n_estimators=1200, reg_alpha=0.7, reg_lambda=2,
subsample=0.7; total time= 1.2min
[CV] END colsample_bytree=0.8, learning_rate=0.03, max_depth=6,
min_child_weight=3, n_estimators=500, reg_alpha=0.5, reg_lambda=2,
subsample=0.9; total time= 1.0min
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=6,
min_child_weight=3, n_estimators=1500, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time= 1.5min
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=4,
min_child_weight=5, n_estimators=800, reg_alpha=0.1, reg_lambda=2,
subsample=0.8; total time=  38.3s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=3,
min_child_weight=1, n_estimators=500, reg_alpha=0, reg_lambda=2, subsample=0.9;
total time=  17.6s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=10,
```

```
min_child_weight=3, n_estimators=1500, reg_alpha=0.5, reg_lambda=2,
subsample=0.7; total time= 3.6min
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=5,
min_child_weight=1, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.9;
total time= 1.1min
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=5,
min_child_weight=1, n_estimators=1500, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time=  59.4s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=1, min_samples_split=4, n_estimators=1500; total time=  49.9s
[CV] END class_weight=balanced, max_depth=12, max_features=log2,
min_samples_leaf=1, min_samples_split=2, n_estimators=500; total time=   6.6s
[CV] END class_weight=balanced, max_depth=12, max_features=log2,
min_samples_leaf=1, min_samples_split=2, n_estimators=500; total time=   6.6s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=log2,
min_samples_leaf=1, min_samples_split=8, n_estimators=1200; total time=  16.7s
[CV] END class_weight=balanced, max_depth=None, max_features=0,
min_samples_leaf=4, min_samples_split=2, n_estimators=500; total time=   0.0s
[CV] END class_weight=balanced, max_depth=None, max_features=0,
min_samples_leaf=4, min_samples_split=2, n_estimators=500; total time=   0.0s
[CV] END class_weight=balanced, max_depth=None, max_features=0,
min_samples_leaf=4, min_samples_split=2, n_estimators=500; total time=   0.0s
[CV] END class_weight=balanced, max_depth=None, max_features=0,
min_samples_leaf=4, min_samples_split=2, n_estimators=500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=8, max_features=3,
min_samples_leaf=2, min_samples_split=8, n_estimators=1000; total time=   6.3s
[CV] END class_weight=balanced, max_depth=16, max_features=3,
min_samples_leaf=4, min_samples_split=5, n_estimators=1200; total time=   8.0s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=2, min_samples_split=4, n_estimators=1200; total time=  33.6s
[CV] END …C=10, gamma=0.01, kernel=rbf; total time= 1.3min
[CV] END …C=1, gamma=0.001, kernel=rbf; total time=  43.2s
[CV] END …C=50, gamma=0.001, kernel=rbf; total time=  39.7s
[CV] END …C=5, gamma=0.005, kernel=rbf; total time= 1.3min
[CV] END …C=0.5, gamma=0.005, kernel=rbf; total time= 1.0min
[CV] END …C=50, gamma=scale, kernel=rbf; total time=  40.1s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=3,
min_child_weight=5, n_estimators=800, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time=  21.4s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=4,
min_child_weight=5, n_estimators=1500, reg_alpha=0.1, reg_lambda=1,
subsample=0.9; total time= 1.2min
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=6,
min_child_weight=1, n_estimators=500, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time=  52.2s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=5,
min_child_weight=5, n_estimators=1200, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time= 1.5min
```

```
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=7,
min_child_weight=5, n_estimators=1200, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time= 1.2min
[CV] END colsample_bytree=0.7, learning_rate=0.1, max_depth=4,
min_child_weight=1, n_estimators=1500, reg_alpha=0, reg_lambda=5, subsample=0.7;
total time= 1.0min
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=5,
min_child_weight=3, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.7;
total time=  53.1s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=3,
min_child_weight=1, n_estimators=1500, reg_alpha=0.5, reg_lambda=1,
subsample=0.8; total time=  44.4s
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=7,
min_child_weight=5, n_estimators=500, reg_alpha=0, reg_lambda=1, subsample=0.7;
total time=  43.8s
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=6,
min_child_weight=5, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.9;
total time=  37.2s
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=6,
min_child_weight=3, n_estimators=1500, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time= 1.5min
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=3,
min_child_weight=3, n_estimators=1500, reg_alpha=0.7, reg_lambda=5,
subsample=0.9; total time=  43.9s
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=4,
min_child_weight=1, n_estimators=500, reg_alpha=0.7, reg_lambda=2,
subsample=0.7; total time=  29.6s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=3,
min_child_weight=5, n_estimators=500, reg_alpha=0.5, reg_lambda=1,
subsample=0.7; total time=  16.7s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=7,
min_child_weight=3, n_estimators=500, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time= 1.1min
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=log2,
min_samples_leaf=4, min_samples_split=2, n_estimators=500; total time=   7.0s
[CV] END class_weight=balanced, max_depth=None, max_features=3,
min_samples_leaf=4, min_samples_split=4, n_estimators=1000; total time=   6.8s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time=  29.8s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time=  30.0s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=3,
min_samples_leaf=2, min_samples_split=4, n_estimators=1000; total time=   7.6s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=sqrt,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time=  15.3s
[CV] END class_weight=balanced, max_depth=None, max_features=sqrt,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time=  27.6s
[CV] END …C=10, gamma=0.0005, kernel=rbf; total time=  38.5s
```

```
[CV] END …C=5, gamma=scale, kernel=rbf; total time= 1.1min
[CV] END …C=1, gamma=0.0005, kernel=rbf; total time=  41.8s
[CV] END …C=0.5, gamma=scale, kernel=rbf; total time=  48.8s
[CV] END …C=1, gamma=scale, kernel=rbf; total time=  50.7s
[CV] END …C=0.5, gamma=0.005, kernel=rbf; total time= 1.1min
[CV] END …C=10, gamma=0.001, kernel=rbf; total time=  30.4s
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=3,
min_child_weight=3, n_estimators=1200, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time=  31.8s
[CV] END colsample_bytree=0.8, learning_rate=0.01, max_depth=6,
min_child_weight=5, n_estimators=500, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time=  49.3s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=5,
min_child_weight=3, n_estimators=1200, reg_alpha=0, reg_lambda=1, subsample=0.8;
total time=  57.3s
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=7,
min_child_weight=1, n_estimators=1200, reg_alpha=0.7, reg_lambda=5,
subsample=0.8; total time= 2.8min
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=4,
min_child_weight=1, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.7;
total time=  40.4s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=6,
min_child_weight=5, n_estimators=1200, reg_alpha=0.7, reg_lambda=1,
subsample=0.9; total time= 1.7min
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=5,
min_child_weight=5, n_estimators=1200, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time=  59.9s
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=6,
min_child_weight=5, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.9;
total time=  34.6s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=5,
min_child_weight=1, n_estimators=800, reg_alpha=0, reg_lambda=5, subsample=0.8;
total time=  59.5s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=10,
min_child_weight=3, n_estimators=500, reg_alpha=0.5, reg_lambda=5,
subsample=0.9; total time= 1.4min
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0.1, reg_lambda=1,
subsample=0.7; total time=  20.9s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=3,
min_child_weight=5, n_estimators=500, reg_alpha=0.5, reg_lambda=1,
subsample=0.7; total time=  15.3s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=7,
min_child_weight=3, n_estimators=500, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time=  52.2s
[CV] END class_weight=balanced, max_depth=8, max_features=log2,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time=   5.3s
[CV] END class_weight=balanced_subsample, max_depth=8, max_features=3,
```

```
min_samples_leaf=1, min_samples_split=8, n_estimators=500; total time=   3.2s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=3,
min_samples_leaf=2, min_samples_split=2, n_estimators=1000; total time=   8.1s
[CV] END class_weight=balanced, max_depth=None, max_features=0,
min_samples_leaf=1, min_samples_split=4, n_estimators=1200; total time=   0.0s
[CV] END class_weight=balanced, max_depth=None, max_features=0,
min_samples_leaf=1, min_samples_split=4, n_estimators=1200; total time=   0.0s
[CV] END class_weight=balanced, max_depth=12, max_features=3,
min_samples_leaf=4, min_samples_split=2, n_estimators=1500; total time=   9.5s
[CV] END class_weight=balanced, max_depth=12, max_features=3,
min_samples_leaf=4, min_samples_split=2, n_estimators=1500; total time=   9.5s
[CV] END class_weight=balanced, max_depth=8, max_features=sqrt,
min_samples_leaf=2, min_samples_split=8, n_estimators=1200; total time=  28.4s
[CV] END class_weight=balanced, max_depth=8, max_features=sqrt,
min_samples_leaf=2, min_samples_split=8, n_estimators=1200; total time=  28.6s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=2, min_samples_split=4, n_estimators=1200; total time=  34.5s
[CV] END …C=10, gamma=0.01, kernel=rbf; total time= 1.3min
[CV] END …C=2, gamma=0.005, kernel=rbf; total time= 1.0min
[CV] END …C=0.5, gamma=scale, kernel=rbf; total time=  34.0s
[CV] END …C=1, gamma=scale, kernel=rbf; total time=  36.4s
[CV] END …C=0.5, gamma=0.01, kernel=rbf; total time= 1.0min
[CV] END …C=0.5, gamma=0.001, kernel=rbf; total time=  29.3s
[CV] END …C=50, gamma=scale, kernel=rbf; total time=  39.6s
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=3,
min_child_weight=3, n_estimators=1200, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time=  32.7s
[CV] END colsample_bytree=0.8, learning_rate=0.01, max_depth=6,
min_child_weight=5, n_estimators=500, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time=  48.0s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=5,
min_child_weight=3, n_estimators=1200, reg_alpha=0, reg_lambda=1, subsample=0.8;
total time=  57.6s
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=7,
min_child_weight=1, n_estimators=1200, reg_alpha=0.7, reg_lambda=5,
subsample=0.8; total time= 2.8min
[CV] END colsample_bytree=0.7, learning_rate=0.1, max_depth=4,
min_child_weight=1, n_estimators=1500, reg_alpha=0, reg_lambda=5, subsample=0.7;
total time= 1.0min
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=5,
min_child_weight=3, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.7;
total time=  50.0s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=10,
min_child_weight=3, n_estimators=1500, reg_alpha=0.5, reg_lambda=2,
subsample=0.7; total time= 3.4min
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=5,
min_child_weight=1, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.9;
total time= 1.1min
```

```
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0.1, reg_lambda=1,
subsample=0.7; total time=  23.8s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=3,
min_child_weight=5, n_estimators=500, reg_alpha=0.5, reg_lambda=1,
subsample=0.7; total time=  18.1s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=7,
min_child_weight=3, n_estimators=500, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time=  55.6s
[CV] END class_weight=balanced, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=sqrt,
min_samples_leaf=1, min_samples_split=8, n_estimators=1000; total time=  33.8s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=sqrt,
min_samples_leaf=4, min_samples_split=5, n_estimators=1000; total time=  29.0s
[CV] END class_weight=balanced, max_depth=8, max_features=sqrt,
min_samples_leaf=2, min_samples_split=8, n_estimators=1200; total time=  29.0s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=0,
min_samples_leaf=4, min_samples_split=5, n_estimators=1500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=0,
min_samples_leaf=4, min_samples_split=5, n_estimators=1500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=0,
min_samples_leaf=4, min_samples_split=5, n_estimators=1500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=0,
min_samples_leaf=4, min_samples_split=5, n_estimators=1500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=0,
min_samples_leaf=4, min_samples_split=5, n_estimators=1500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=2, min_samples_split=4, n_estimators=1200; total time=  35.0s
[CV] END …C=10, gamma=0.01, kernel=rbf; total time= 1.3min
[CV] END …C=1, gamma=0.001, kernel=rbf; total time=  43.4s
[CV] END …C=50, gamma=0.001, kernel=rbf; total time=  41.2s
[CV] END …C=20, gamma=0.005, kernel=rbf; total time= 1.3min
[CV] END …C=0.5, gamma=0.005, kernel=rbf; total time= 1.1min
[CV] END …C=10, gamma=0.001, kernel=rbf; total time=  29.3s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=3,
min_child_weight=5, n_estimators=800, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time=  24.3s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=4,
min_child_weight=5, n_estimators=1500, reg_alpha=0.1, reg_lambda=1,
subsample=0.9; total time= 1.1min
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=6,
min_child_weight=1, n_estimators=500, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time=  52.5s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=5,
min_child_weight=5, n_estimators=1200, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time= 1.5min
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=7,
```

```
min_child_weight=5, n_estimators=1200, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time= 1.1min
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=4,
min_child_weight=1, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.7;
total time=  39.6s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=6,
min_child_weight=5, n_estimators=1200, reg_alpha=0.7, reg_lambda=1,
subsample=0.9; total time= 1.8min
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=5,
min_child_weight=5, n_estimators=1200, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time= 1.0min
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0, reg_lambda=1, subsample=0.8;
total time=  24.0s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=5,
min_child_weight=1, n_estimators=800, reg_alpha=0, reg_lambda=5, subsample=0.8;
total time= 1.0min
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=10,
min_child_weight=3, n_estimators=500, reg_alpha=0.5, reg_lambda=5,
subsample=0.9; total time= 1.6min
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0.1, reg_lambda=1,
subsample=0.7; total time=  22.9s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=3,
min_child_weight=5, n_estimators=500, reg_alpha=0.5, reg_lambda=1,
subsample=0.7; total time=  16.8s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=7,
min_child_weight=3, n_estimators=500, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time=  57.8s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=log2,
min_samples_leaf=4, min_samples_split=2, n_estimators=500; total time=   6.7s
[CV] END class_weight=balanced, max_depth=None, max_features=3,
min_samples_leaf=4, min_samples_split=4, n_estimators=1000; total time=   6.7s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=sqrt,
min_samples_leaf=1, min_samples_split=4, n_estimators=500; total time=  15.3s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=sqrt,
min_samples_leaf=1, min_samples_split=4, n_estimators=500; total time=  15.3s
[CV] END class_weight=balanced, max_depth=8, max_features=log2,
min_samples_leaf=1, min_samples_split=2, n_estimators=1000; total time=  10.7s
[CV] END class_weight=balanced, max_depth=8, max_features=log2,
min_samples_leaf=1, min_samples_split=2, n_estimators=1000; total time=  10.7s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=log2,
min_samples_leaf=1, min_samples_split=8, n_estimators=1200; total time=  16.8s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=sqrt,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time=  15.1s
[CV] END class_weight=balanced, max_depth=None, max_features=log2,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time=  12.6s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=3,
```

```
min_samples_leaf=1, min_samples_split=5, n_estimators=1000; total time=   8.0s
[CV] END …C=20, gamma=0.01, kernel=rbf; total time= 1.3min
[CV] END …C=2, gamma=0.005, kernel=rbf; total time= 1.3min
[CV] END …C=0.5, gamma=0.0005, kernel=rbf; total time=  47.1s
[CV] END …C=1, gamma=scale, kernel=rbf; total time=  48.0s
[CV] END …C=50, gamma=0.005, kernel=rbf; total time= 1.1min
[CV] END …C=50, gamma=0.01, kernel=rbf; total time=  34.5s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=4,
min_child_weight=5, n_estimators=1500, reg_alpha=0.7, reg_lambda=2,
subsample=0.9; total time= 1.0min
[CV] END colsample_bytree=0.8, learning_rate=0.01, max_depth=5,
min_child_weight=1, n_estimators=800, reg_alpha=0.1, reg_lambda=5,
subsample=0.8; total time= 1.2min
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=6,
min_child_weight=1, n_estimators=1200, reg_alpha=0.7, reg_lambda=2,
subsample=0.7; total time= 1.2min
[CV] END colsample_bytree=0.8, learning_rate=0.03, max_depth=6,
min_child_weight=3, n_estimators=500, reg_alpha=0.5, reg_lambda=2,
subsample=0.9; total time=  55.8s
[CV] END colsample_bytree=0.8, learning_rate=0.01, max_depth=5,
min_child_weight=1, n_estimators=1500, reg_alpha=0.5, reg_lambda=2,
subsample=0.9; total time= 2.3min
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=3,
min_child_weight=1, n_estimators=500, reg_alpha=0, reg_lambda=2, subsample=0.9;
total time=  17.5s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=10,
min_child_weight=3, n_estimators=1500, reg_alpha=0.5, reg_lambda=2,
subsample=0.7; total time= 3.6min
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=5,
min_child_weight=1, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.9;
total time= 1.1min
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=5,
min_child_weight=1, n_estimators=1500, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time=  56.9s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=1, min_samples_split=4, n_estimators=1500; total time=  50.0s
[CV] END class_weight=balanced, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=4, n_estimators=1000; total time=   0.0s
[CV] END class_weight=balanced, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=4, n_estimators=1000; total time=   0.0s
[CV] END class_weight=balanced, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=4, n_estimators=1000; total time=   0.0s
[CV] END class_weight=balanced, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=4, n_estimators=1000; total time=   0.0s
[CV] END class_weight=balanced, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=4, n_estimators=1000; total time=   0.0s
[CV] END class_weight=balanced, max_depth=12, max_features=3,
min_samples_leaf=2, min_samples_split=2, n_estimators=1000; total time=   6.7s
```

```
[CV] END class_weight=balanced, max_depth=12, max_features=3,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time=   6.4s
[CV] END class_weight=balanced, max_depth=12, max_features=3,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time=   6.4s
[CV] END class_weight=balanced, max_depth=8, max_features=log2,
min_samples_leaf=2, min_samples_split=4, n_estimators=1200; total time=  12.8s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=sqrt,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time=  15.0s
[CV] END class_weight=balanced, max_depth=None, max_features=log2,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time=  12.8s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=3,
min_samples_leaf=1, min_samples_split=5, n_estimators=1000; total time=   8.0s
[CV] END …C=20, gamma=0.01, kernel=rbf; total time= 1.4min
[CV] END …C=5, gamma=0.01, kernel=rbf; total time= 1.4min
[CV] END …C=5, gamma=0.005, kernel=rbf; total time= 1.3min
[CV] END …C=0.5, gamma=0.005, kernel=rbf; total time= 1.1min
[CV] END …C=10, gamma=0.001, kernel=rbf; total time=  34.9s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=3,
min_child_weight=5, n_estimators=800, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time=  20.0s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=5,
min_child_weight=5, n_estimators=1500, reg_alpha=0.7, reg_lambda=2,
subsample=0.7; total time=  54.6s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=6,
min_child_weight=1, n_estimators=1200, reg_alpha=0.1, reg_lambda=2,
subsample=0.9; total time= 3.1min
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=6,
min_child_weight=3, n_estimators=1500, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time= 1.5min
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=4,
min_child_weight=5, n_estimators=800, reg_alpha=0.1, reg_lambda=2,
subsample=0.8; total time=  39.3s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=6,
min_child_weight=5, n_estimators=1200, reg_alpha=0.5, reg_lambda=1,
subsample=0.8; total time=  48.9s
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=10,
min_child_weight=5, n_estimators=500, reg_alpha=0.1, reg_lambda=1,
subsample=0.9; total time= 1.1min
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=6,
min_child_weight=5, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.9;
total time=  36.9s
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=6,
min_child_weight=3, n_estimators=1500, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time= 1.5min
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=3,
min_child_weight=3, n_estimators=1500, reg_alpha=0.7, reg_lambda=5,
subsample=0.9; total time=  41.1s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=10,
```

```
min_child_weight=1, n_estimators=800, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time= 1.9min
[CV] END class_weight=balanced, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=sqrt,
min_samples_leaf=1, min_samples_split=8, n_estimators=1000; total time=  33.3s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=sqrt,
min_samples_leaf=4, min_samples_split=5, n_estimators=1000; total time=  28.8s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=sqrt,
min_samples_leaf=4, min_samples_split=5, n_estimators=1000; total time=  29.2s
[CV] END class_weight=balanced, max_depth=None, max_features=sqrt,
min_samples_leaf=1, min_samples_split=5, n_estimators=1000; total time=  31.7s
[CV] END …C=2, gamma=0.001, kernel=rbf; total time=  43.5s
[CV] END …C=20, gamma=0.0005, kernel=rbf; total time=  36.8s
[CV] END …C=1, gamma=0.001, kernel=rbf; total time=  42.9s
[CV] END …C=50, gamma=0.001, kernel=rbf; total time=  40.8s
[CV] END …C=5, gamma=0.005, kernel=rbf; total time= 1.3min
[CV] END …C=0.5, gamma=0.005, kernel=rbf; total time= 1.1min
[CV] END …C=10, gamma=0.001, kernel=rbf; total time=  34.6s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=3,
min_child_weight=5, n_estimators=800, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time=  21.8s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=4,
min_child_weight=5, n_estimators=1500, reg_alpha=0.1, reg_lambda=1,
subsample=0.9; total time= 1.1min
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=6,
min_child_weight=1, n_estimators=500, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time=  50.8s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=5,
min_child_weight=5, n_estimators=1200, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time= 1.4min
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=7,
min_child_weight=5, n_estimators=1200, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time= 1.1min
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=4,
min_child_weight=1, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.7;
total time=  39.9s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=6,
min_child_weight=5, n_estimators=1200, reg_alpha=0.7, reg_lambda=1,
subsample=0.9; total time= 1.7min
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=10,
min_child_weight=5, n_estimators=500, reg_alpha=0.1, reg_lambda=1,
subsample=0.9; total time= 1.1min
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=6,
min_child_weight=5, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.9;
total time=  40.0s
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=6,
min_child_weight=3, n_estimators=1500, reg_alpha=0.1, reg_lambda=2,
```

```
subsample=0.7; total time= 1.5min
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=3,
min_child_weight=3, n_estimators=1500, reg_alpha=0.7, reg_lambda=5,
subsample=0.9; total time=  40.7s
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=4,
min_child_weight=1, n_estimators=500, reg_alpha=0.7, reg_lambda=2,
subsample=0.7; total time=  27.4s
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=3,
min_child_weight=5, n_estimators=1500, reg_alpha=0.1, reg_lambda=5,
subsample=0.9; total time=  42.2s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=4,
min_child_weight=5, n_estimators=800, reg_alpha=0.7, reg_lambda=5,
subsample=0.8; total time=  23.6s
[CV] END class_weight=balanced, max_depth=8, max_features=log2,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time=   5.4s
[CV] END class_weight=balanced_subsample, max_depth=8, max_features=3,
min_samples_leaf=1, min_samples_split=8, n_estimators=500; total time=   3.2s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=3,
min_samples_leaf=2, min_samples_split=2, n_estimators=1000; total time=   8.1s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=1, min_samples_split=4, n_estimators=1000; total time=  32.9s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=1, min_samples_split=4, n_estimators=1000; total time=  33.5s
[CV] END class_weight=balanced, max_depth=16, max_features=3,
min_samples_leaf=4, min_samples_split=5, n_estimators=1200; total time=   8.1s
[CV] END class_weight=balanced, max_depth=None, max_features=sqrt,
min_samples_leaf=1, min_samples_split=5, n_estimators=1000; total time=  32.2s
[CV] END …C=10, gamma=0.0005, kernel=rbf; total time=  25.2s
[CV] END …C=5, gamma=scale, kernel=rbf; total time=  56.8s
[CV] END …C=1, gamma=0.001, kernel=rbf; total time=  43.5s
[CV] END …C=50, gamma=0.001, kernel=rbf; total time=  41.2s
[CV] END …C=20, gamma=0.005, kernel=rbf; total time= 1.3min
[CV] END …C=50, gamma=0.005, kernel=rbf; total time= 1.3min
[CV] END …C=50, gamma=0.01, kernel=rbf; total time=  26.9s
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=3,
min_child_weight=3, n_estimators=1200, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time=  32.7s
[CV] END colsample_bytree=0.8, learning_rate=0.01, max_depth=6,
min_child_weight=5, n_estimators=500, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time=  47.0s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=5,
min_child_weight=3, n_estimators=1200, reg_alpha=0, reg_lambda=1, subsample=0.8;
total time=  57.0s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=6,
min_child_weight=1, n_estimators=1200, reg_alpha=0.7, reg_lambda=2,
subsample=0.7; total time= 1.2min
[CV] END colsample_bytree=0.8, learning_rate=0.03, max_depth=6,
min_child_weight=3, n_estimators=500, reg_alpha=0.5, reg_lambda=2,
```

```
subsample=0.9; total time=  58.9s
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=6,
min_child_weight=3, n_estimators=1500, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time= 1.5min
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=4,
min_child_weight=5, n_estimators=800, reg_alpha=0.1, reg_lambda=2,
subsample=0.8; total time=  39.8s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=3,
min_child_weight=1, n_estimators=500, reg_alpha=0, reg_lambda=2, subsample=0.9;
total time=  17.9s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=10,
min_child_weight=3, n_estimators=1500, reg_alpha=0.5, reg_lambda=2,
subsample=0.7; total time= 3.6min
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=5,
min_child_weight=1, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.9;
total time= 1.1min
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=5,
min_child_weight=1, n_estimators=1500, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time= 1.0min
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=1, min_samples_split=4, n_estimators=1500; total time=  49.4s
[CV] END class_weight=balanced, max_depth=8, max_features=log2,
min_samples_leaf=1, min_samples_split=2, n_estimators=1000; total time=  10.7s
[CV] END class_weight=balanced, max_depth=12, max_features=log2,
min_samples_leaf=1, min_samples_split=2, n_estimators=500; total time=   6.7s
[CV] END class_weight=balanced, max_depth=8, max_features=log2,
min_samples_leaf=2, min_samples_split=4, n_estimators=1200; total time=  12.8s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=3,
min_samples_leaf=2, min_samples_split=4, n_estimators=1000; total time=   7.6s
[CV] END class_weight=balanced, max_depth=16, max_features=3,
min_samples_leaf=4, min_samples_split=8, n_estimators=1500; total time=   9.9s
[CV] END class_weight=balanced, max_depth=None, max_features=log2,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time=  12.7s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=3,
min_samples_leaf=1, min_samples_split=5, n_estimators=1000; total time=   7.9s
[CV] END …C=20, gamma=0.01, kernel=rbf; total time= 1.4min
[CV] END …C=5, gamma=0.01, kernel=rbf; total time= 1.4min
[CV] END …C=20, gamma=0.005, kernel=rbf; total time= 1.3min
[CV] END …C=50, gamma=0.005, kernel=rbf; total time= 1.3min
[CV] END …C=50, gamma=0.01, kernel=rbf; total time=  26.2s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0.5, reg_lambda=5,
subsample=0.9; total time=  22.3s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=4,
min_child_weight=5, n_estimators=1500, reg_alpha=0.1, reg_lambda=1,
subsample=0.9; total time= 1.2min
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=6,
min_child_weight=1, n_estimators=500, reg_alpha=0, reg_lambda=1, subsample=0.9;
```

```
total time=  54.5s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=5,
min_child_weight=5, n_estimators=1200, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time= 1.5min
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=7,
min_child_weight=5, n_estimators=1200, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time= 1.1min
[CV] END colsample_bytree=0.7, learning_rate=0.1, max_depth=4,
min_child_weight=1, n_estimators=1500, reg_alpha=0, reg_lambda=5, subsample=0.7;
total time= 1.1min
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=5,
min_child_weight=3, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.7;
total time=  51.8s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=3,
min_child_weight=1, n_estimators=1500, reg_alpha=0.5, reg_lambda=1,
subsample=0.8; total time=  52.1s
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=7,
min_child_weight=5, n_estimators=500, reg_alpha=0, reg_lambda=1, subsample=0.7;
total time=  43.1s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0, reg_lambda=1, subsample=0.8;
total time=  25.4s
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=6,
min_child_weight=3, n_estimators=1500, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time= 1.5min
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=5,
min_child_weight=1, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.9;
total time= 1.1min
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=5,
min_child_weight=1, n_estimators=1500, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time= 1.1min
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=1, min_samples_split=4, n_estimators=1500; total time=  50.2s
[CV] END class_weight=balanced, max_depth=12, max_features=3,
min_samples_leaf=2, min_samples_split=2, n_estimators=1000; total time=   6.6s
[CV] END class_weight=balanced, max_depth=12, max_features=3,
min_samples_leaf=2, min_samples_split=2, n_estimators=1000; total time=   6.7s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=log2,
min_samples_leaf=1, min_samples_split=8, n_estimators=1200; total time=  16.8s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=sqrt,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time=  15.2s
[CV] END class_weight=balanced, max_depth=None, max_features=sqrt,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time=  28.0s
[CV] END …C=10, gamma=0.0005, kernel=rbf; total time=  37.6s
[CV] END …C=5, gamma=scale, kernel=rbf; total time= 1.1min
[CV] END …C=5, gamma=0.01, kernel=rbf; total time= 1.4min
[CV] END …C=1, gamma=scale, kernel=rbf; total time=  50.2s
[CV] END …C=0.5, gamma=0.01, kernel=rbf; total time= 1.2min
```

```
[CV] END …C=50, gamma=0.01, kernel=rbf; total time=  38.6s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0.5, reg_lambda=5,
subsample=0.9; total time=  20.4s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=5,
min_child_weight=5, n_estimators=1500, reg_alpha=0.7, reg_lambda=2,
subsample=0.7; total time=  52.7s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=6,
min_child_weight=1, n_estimators=1200, reg_alpha=0.1, reg_lambda=2,
subsample=0.9; total time= 2.9min
[CV] END colsample_bytree=0.8, learning_rate=0.01, max_depth=5,
min_child_weight=1, n_estimators=1500, reg_alpha=0.5, reg_lambda=2,
subsample=0.9; total time= 2.2min
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=6,
min_child_weight=5, n_estimators=1200, reg_alpha=0.5, reg_lambda=1,
subsample=0.8; total time=  49.1s
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=10,
min_child_weight=5, n_estimators=500, reg_alpha=0.1, reg_lambda=1,
subsample=0.9; total time= 1.0min
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0.5, reg_lambda=2,
subsample=0.9; total time=  30.1s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=5,
min_child_weight=5, n_estimators=800, reg_alpha=0.5, reg_lambda=5,
subsample=0.7; total time=  51.8s
[CV] END colsample_bytree=0.7, learning_rate=0.1, max_depth=5,
min_child_weight=1, n_estimators=500, reg_alpha=0.5, reg_lambda=5,
subsample=0.9; total time=  45.3s
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=4,
min_child_weight=5, n_estimators=1200, reg_alpha=0, reg_lambda=5, subsample=0.7;
total time=  40.2s
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=3,
min_child_weight=3, n_estimators=500, reg_alpha=0.1, reg_lambda=1,
subsample=0.9; total time=  13.1s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=10,
min_child_weight=1, n_estimators=800, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time= 1.9min
[CV] END class_weight=balanced, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=sqrt,
min_samples_leaf=1, min_samples_split=8, n_estimators=1000; total time=  33.2s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=3,
min_samples_leaf=4, min_samples_split=5, n_estimators=1500; total time=  11.4s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=3,
min_samples_leaf=4, min_samples_split=5, n_estimators=1500; total time=  11.6s
[CV] END class_weight=balanced, max_depth=12, max_features=3,
min_samples_leaf=2, min_samples_split=2, n_estimators=1000; total time=   6.8s
[CV] END class_weight=balanced, max_depth=12, max_features=3,
```

```
min_samples_leaf=2, min_samples_split=2, n_estimators=1000; total time=   6.8s
[CV] END class_weight=balanced, max_depth=8, max_features=log2,
min_samples_leaf=2, min_samples_split=4, n_estimators=1200; total time=  12.9s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=sqrt,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time=  15.2s
[CV] END class_weight=balanced, max_depth=None, max_features=log2,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time=  12.3s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=3,
min_samples_leaf=1, min_samples_split=5, n_estimators=1000; total time=   8.2s
[CV] END …C=20, gamma=0.01, kernel=rbf; total time= 1.3min
[CV] END …C=5, gamma=0.0005, kernel=rbf; total time=  25.4s
[CV] END …C=1, gamma=0.0005, kernel=rbf; total time=  29.9s
[CV] END …C=0.5, gamma=scale, kernel=rbf; total time=  36.7s
[CV] END …C=20, gamma=0.005, kernel=rbf; total time= 1.1min
[CV] END …C=0.5, gamma=0.01, kernel=rbf; total time= 1.0min
[CV] END …C=50, gamma=scale, kernel=rbf; total time=  46.2s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=4,
min_child_weight=5, n_estimators=1500, reg_alpha=0.7, reg_lambda=2,
subsample=0.9; total time= 1.1min
[CV] END colsample_bytree=0.8, learning_rate=0.01, max_depth=5,
min_child_weight=1, n_estimators=800, reg_alpha=0.1, reg_lambda=5,
subsample=0.8; total time= 1.3min
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=7,
min_child_weight=1, n_estimators=1200, reg_alpha=0.7, reg_lambda=5,
subsample=0.8; total time= 2.8min
[CV] END colsample_bytree=0.7, learning_rate=0.1, max_depth=4,
min_child_weight=1, n_estimators=1500, reg_alpha=0, reg_lambda=5, subsample=0.7;
total time= 1.0min
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=5,
min_child_weight=3, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.7;
total time=  52.4s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=3,
min_child_weight=1, n_estimators=1500, reg_alpha=0.5, reg_lambda=1,
subsample=0.8; total time=  49.7s
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=7,
min_child_weight=5, n_estimators=500, reg_alpha=0, reg_lambda=1, subsample=0.7;
total time=  40.7s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0, reg_lambda=1, subsample=0.8;
total time=  24.5s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=5,
min_child_weight=1, n_estimators=800, reg_alpha=0, reg_lambda=5, subsample=0.8;
total time=  58.6s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=10,
min_child_weight=3, n_estimators=500, reg_alpha=0.5, reg_lambda=5,
subsample=0.9; total time= 1.4min
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0.1, reg_lambda=1,
```

```
subsample=0.7; total time=  21.9s
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=3,
min_child_weight=5, n_estimators=1500, reg_alpha=0.1, reg_lambda=5,
subsample=0.9; total time=  40.0s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=4,
min_child_weight=5, n_estimators=800, reg_alpha=0.7, reg_lambda=5,
subsample=0.8; total time=  29.1s
[CV] END class_weight=balanced, max_depth=8, max_features=log2,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time=   5.4s
[CV] END class_weight=balanced_subsample, max_depth=8, max_features=3,
min_samples_leaf=1, min_samples_split=8, n_estimators=500; total time=   3.2s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=3,
min_samples_leaf=2, min_samples_split=2, n_estimators=1000; total time=   8.1s
[CV] END class_weight=balanced, max_depth=12, max_features=3,
min_samples_leaf=4, min_samples_split=2, n_estimators=1500; total time=   9.4s
[CV] END class_weight=balanced, max_depth=12, max_features=3,
min_samples_leaf=4, min_samples_split=2, n_estimators=1500; total time=   9.4s
[CV] END class_weight=balanced, max_depth=8, max_features=sqrt,
min_samples_leaf=2, min_samples_split=8, n_estimators=1200; total time=  28.4s
[CV] END class_weight=balanced, max_depth=8, max_features=sqrt,
min_samples_leaf=2, min_samples_split=8, n_estimators=1200; total time=  28.8s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=2, min_samples_split=4, n_estimators=1200; total time=  34.4s
[CV] END …C=10, gamma=0.01, kernel=rbf; total time= 1.4min
[CV] END …C=1, gamma=0.001, kernel=rbf; total time=  43.8s
[CV] END …C=50, gamma=0.001, kernel=rbf; total time=  41.5s
[CV] END …C=20, gamma=0.005, kernel=rbf; total time= 1.3min
[CV] END …C=50, gamma=0.005, kernel=rbf; total time= 1.3min
[CV] END …C=50, gamma=0.01, kernel=rbf; total time=  26.9s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0.5, reg_lambda=5,
subsample=0.9; total time=  21.7s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=4,
min_child_weight=5, n_estimators=1500, reg_alpha=0.1, reg_lambda=1,
subsample=0.9; total time= 1.1min
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=6,
min_child_weight=1, n_estimators=500, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time=  50.4s
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=7,
min_child_weight=1, n_estimators=1200, reg_alpha=0.7, reg_lambda=5,
subsample=0.8; total time= 2.8min
[CV] END colsample_bytree=0.7, learning_rate=0.1, max_depth=4,
min_child_weight=1, n_estimators=1500, reg_alpha=0, reg_lambda=5, subsample=0.7;
total time= 1.1min
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=5,
min_child_weight=3, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.7;
total time=  49.8s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=3,
```

```
min_child_weight=1, n_estimators=1500, reg_alpha=0.5, reg_lambda=1,
subsample=0.8; total time=  55.0s
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=7,
min_child_weight=5, n_estimators=500, reg_alpha=0, reg_lambda=1, subsample=0.7;
total time=  40.5s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0, reg_lambda=1, subsample=0.8;
total time=  24.8s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=5,
min_child_weight=1, n_estimators=800, reg_alpha=0, reg_lambda=5, subsample=0.8;
total time= 1.0min
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=10,
min_child_weight=3, n_estimators=500, reg_alpha=0.5, reg_lambda=5,
subsample=0.9; total time= 1.3min
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0.1, reg_lambda=1,
subsample=0.7; total time=  21.1s
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=3,
min_child_weight=5, n_estimators=1500, reg_alpha=0.1, reg_lambda=5,
subsample=0.9; total time=  40.5s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=4,
min_child_weight=5, n_estimators=800, reg_alpha=0.7, reg_lambda=5,
subsample=0.8; total time=  33.2s
[CV] END class_weight=balanced, max_depth=8, max_features=log2,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time=   5.3s
[CV] END class_weight=balanced_subsample, max_depth=8, max_features=3,
min_samples_leaf=1, min_samples_split=8, n_estimators=500; total time=   3.2s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=3,
min_samples_leaf=2, min_samples_split=2, n_estimators=1000; total time=   8.1s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=0,
min_samples_leaf=4, min_samples_split=8, n_estimators=1500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=0,
min_samples_leaf=4, min_samples_split=8, n_estimators=1500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=0,
min_samples_leaf=4, min_samples_split=8, n_estimators=1500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=0,
min_samples_leaf=4, min_samples_split=8, n_estimators=1500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=1, min_samples_split=4, n_estimators=1000; total time=  33.2s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=1, min_samples_split=4, n_estimators=1000; total time=  33.3s
[CV] END class_weight=balanced, max_depth=16, max_features=3,
min_samples_leaf=4, min_samples_split=5, n_estimators=1200; total time=   8.1s
[CV] END class_weight=balanced, max_depth=None, max_features=sqrt,
min_samples_leaf=1, min_samples_split=5, n_estimators=1000; total time=  32.2s
[CV] END …C=10, gamma=0.0005, kernel=rbf; total time=  24.7s
[CV] END …C=5, gamma=scale, kernel=rbf; total time=  47.1s
[CV] END …C=5, gamma=0.0005, kernel=rbf; total time=  24.8s
```

```
[CV] END …C=5, gamma=0.01, kernel=rbf; total time= 1.1min
[CV] END …C=5, gamma=0.005, kernel=rbf; total time= 1.1min
[CV] END …C=0.5, gamma=0.01, kernel=rbf; total time= 1.1min
[CV] END …C=50, gamma=scale, kernel=rbf; total time=  44.4s
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=3,
min_child_weight=3, n_estimators=1200, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time=  34.4s
[CV] END colsample_bytree=0.8, learning_rate=0.01, max_depth=6,
min_child_weight=5, n_estimators=500, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time=  47.8s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=5,
min_child_weight=3, n_estimators=1200, reg_alpha=0, reg_lambda=1, subsample=0.8;
total time=  56.7s
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=7,
min_child_weight=1, n_estimators=1200, reg_alpha=0.7, reg_lambda=5,
subsample=0.8; total time= 2.7min
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=4,
min_child_weight=1, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.7;
total time=  40.6s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=6,
min_child_weight=5, n_estimators=1200, reg_alpha=0.7, reg_lambda=1,
subsample=0.9; total time= 1.8min
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=5,
min_child_weight=5, n_estimators=1200, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time=  58.2s
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=6,
min_child_weight=5, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.9;
total time=  38.4s
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=6,
min_child_weight=3, n_estimators=1500, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time= 1.4min
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=3,
min_child_weight=3, n_estimators=1500, reg_alpha=0.7, reg_lambda=5,
subsample=0.9; total time=  44.1s
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=4,
min_child_weight=1, n_estimators=500, reg_alpha=0.7, reg_lambda=2,
subsample=0.7; total time=  29.7s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=3,
min_child_weight=5, n_estimators=500, reg_alpha=0.5, reg_lambda=1,
subsample=0.7; total time=  16.8s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=7,
min_child_weight=3, n_estimators=500, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time= 1.0min
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=log2,
min_samples_leaf=4, min_samples_split=2, n_estimators=500; total time=   6.8s
[CV] END class_weight=balanced, max_depth=None, max_features=3,
min_samples_leaf=4, min_samples_split=4, n_estimators=1000; total time=   6.8s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
```

```
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time=  29.6s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time=  29.6s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=3,
min_samples_leaf=2, min_samples_split=4, n_estimators=1000; total time=   7.5s
[CV] END class_weight=balanced_subsample, max_depth=8, max_features=3,
min_samples_leaf=2, min_samples_split=8, n_estimators=1000; total time=   6.4s
[CV] END class_weight=balanced, max_depth=16, max_features=3,
min_samples_leaf=4, min_samples_split=8, n_estimators=1500; total time=  10.0s
[CV] END class_weight=balanced, max_depth=None, max_features=sqrt,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time=  26.5s
[CV] END …C=2, gamma=0.001, kernel=rbf; total time=  41.8s
[CV] END …C=20, gamma=0.0005, kernel=rbf; total time=  34.9s
[CV] END …C=5, gamma=0.0005, kernel=rbf; total time=  38.5s
[CV] END …C=1, gamma=0.0005, kernel=rbf; total time=  42.3s
[CV] END …C=0.5, gamma=0.0005, kernel=rbf; total time=  47.8s
[CV] END …C=2, gamma=0.01, kernel=rbf; total time= 1.4min
[CV] END …C=50, gamma=scale, kernel=rbf; total time=  52.3s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0.5, reg_lambda=5,
subsample=0.9; total time=  21.0s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=5,
min_child_weight=5, n_estimators=1500, reg_alpha=0.7, reg_lambda=2,
subsample=0.7; total time=  55.9s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=6,
min_child_weight=1, n_estimators=1200, reg_alpha=0.1, reg_lambda=2,
subsample=0.9; total time= 3.1min
[CV] END colsample_bytree=0.8, learning_rate=0.01, max_depth=5,
min_child_weight=1, n_estimators=1500, reg_alpha=0.5, reg_lambda=2,
subsample=0.9; total time= 2.4min
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=3,
min_child_weight=1, n_estimators=500, reg_alpha=0, reg_lambda=2, subsample=0.9;
total time=  17.6s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=3,
min_child_weight=1, n_estimators=1500, reg_alpha=0.5, reg_lambda=1,
subsample=0.8; total time=  49.8s
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=7,
min_child_weight=5, n_estimators=500, reg_alpha=0, reg_lambda=1, subsample=0.7;
total time=  40.6s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0, reg_lambda=1, subsample=0.8;
total time=  22.5s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=5,
min_child_weight=1, n_estimators=800, reg_alpha=0, reg_lambda=5, subsample=0.8;
total time=  58.4s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=10,
min_child_weight=3, n_estimators=500, reg_alpha=0.5, reg_lambda=5,
subsample=0.9; total time= 1.3min
```

```
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=10,
min_child_weight=1, n_estimators=800, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time= 1.9min
[CV] END class_weight=balanced, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=sqrt,
min_samples_leaf=1, min_samples_split=8, n_estimators=1000; total time=  33.3s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=3,
min_samples_leaf=4, min_samples_split=5, n_estimators=1500; total time=  11.5s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=3,
min_samples_leaf=4, min_samples_split=5, n_estimators=1500; total time=  11.7s
[CV] END class_weight=balanced, max_depth=8, max_features=0, min_samples_leaf=2,
min_samples_split=8, n_estimators=500; total time=   0.0s
[CV] END class_weight=balanced, max_depth=8, max_features=0, min_samples_leaf=2,
min_samples_split=8, n_estimators=500; total time=   0.0s
[CV] END class_weight=balanced, max_depth=8, max_features=0, min_samples_leaf=2,
min_samples_split=8, n_estimators=500; total time=   0.0s
[CV] END class_weight=balanced, max_depth=8, max_features=0, min_samples_leaf=2,
min_samples_split=8, n_estimators=500; total time=   0.0s
[CV] END class_weight=balanced, max_depth=8, max_features=0, min_samples_leaf=2,
min_samples_split=8, n_estimators=500; total time=   0.0s
[CV] END class_weight=balanced, max_depth=12, max_features=3,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time=   6.6s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=log2,
min_samples_leaf=1, min_samples_split=8, n_estimators=1200; total time=  17.1s
[CV] END class_weight=balanced_subsample, max_depth=8, max_features=3,
min_samples_leaf=2, min_samples_split=8, n_estimators=1000; total time=   6.7s
[CV] END class_weight=balanced, max_depth=16, max_features=3,
min_samples_leaf=4, min_samples_split=8, n_estimators=1500; total time=  10.2s
[CV] END class_weight=balanced, max_depth=None, max_features=log2,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time=  12.5s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=2, n_estimators=1200; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=2, n_estimators=1200; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=2, n_estimators=1200; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=2, n_estimators=1200; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=2, n_estimators=1200; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=3,
min_samples_leaf=1, min_samples_split=5, n_estimators=1000; total time=   8.3s
[CV] END …C=20, gamma=0.01, kernel=rbf; total time= 1.3min
[CV] END …C=5, gamma=0.0005, kernel=rbf; total time=  37.3s
[CV] END …C=1, gamma=0.0005, kernel=rbf; total time=  43.0s
[CV] END …C=0.5, gamma=0.0005, kernel=rbf; total time=  47.1s
[CV] END …C=2, gamma=0.01, kernel=rbf; total time= 1.4min
```

```
[CV] END …C=0.5, gamma=0.001, kernel=rbf; total time=  44.4s

PermutationExplainer explainer: 201it [1:41:01, 30.31s/it]


Shap SHAP: (200, 445)
[SVM] Gráfico salvo com sucesso em 'pipelineAD/svm_importancia_permutacao.png'!


Finalizado fase de treinamento em: 17:42:00

Tempo decorrido: 2:06:03.405425

Fim criação/treino dos modelos


<Figure size 500x500 with 0 Axes>

<Figure size 500x500 with 0 Axes>

<Figure size 500x500 with 0 Axes>
```

[12]:
```python
'''
Avaliação e Validação dos modelos
'''
print("\n\nInciando Validação dos modelos\n")
# verificar importância dos atributos/feature para o Random Forest
importancia = pd.DataFrame({"feature":columns,"importance":rf_model.
 ↪feature_importances_})
importancia = importancia.sort_values("importance",ascending = False)
print("\nTop 20 features: Random Forest")
print(importancia.head(20))
# GRÁFICO
top = importancia.head(20)
plt.figure(figsize=(10,8))
plt.barh( top["feature"][::-1], top["importance"][::-1] )
plt.xlabel("Importância")
plt.ylabel("Feature")
# plt.title("Top 20 Features - Random Forest")
plt.tight_layout()
plt.savefig(
    f"{OUTPUT_DIR}/rf_importancia_variaveis.png",
    bbox_inches="tight"
)
plt.close()

# verificar importância dos atributos/feature para o XGBoost
importancia = pd.DataFrame({"feature":columns, "importance":xgb_model.
 ↪feature_importances_})
importancia = importancia.sort_values("importance", ascending = False)
```

```
print("\nTop 20 features: XGBoost")
print(importancia.head(20))
# GRÁFICO
top = importancia.head(20)
plt.figure(figsize=(10,8))
plt.barh(top["feature"][::-1], top["importance"][::-1])
plt.xlabel("Importância")
plt.ylabel("Feature")
# plt.title("Top 20 Features - XGBoost")
plt.tight_layout()
plt.savefig(
    f"{OUTPUT_DIR}/xgb_importancia_variaveis.png",
    bbox_inches="tight"
)
plt.close()

print("\nCOMPARAÇÃO MODELOS")
print(pd.DataFrame(results))

results_df = pd.DataFrame(results)
results_df.to_csv(
        f"{OUTPUT_DIR}/metricas_comparacao.csv",
        index=False
    )

metrics = [
    "ROC_AUC",
    "F1",
    "Precision",
    "Recall",
    "Balanced_Accuracy"
]

for metric in metrics:
    plt.figure(figsize=(8,5))
    plt.bar(
        results_df["Modelo"],
        results_df[metric]
    )

    plt.ylim(0, 1)
    plt.ylabel(metric)
    # plt.title(f"Comparação dos Modelos - {metric}")
    for i, v in enumerate(results_df[metric]):
        plt.text(i, v + 0.01, f"{v:.3f}", ha='center')

    plt.savefig(
```

```
        f"{OUTPUT_DIR}/comparacao_{metric}.png",
        bbox_inches="tight"
    )
    plt.close()

modelos = {
    "Random Forest":rf_model,
    "XGBoost":xgb_model,
    "SVM":svm_model
    }
plt.figure(figsize=(7,7))
for nome, model in modelos.items():
    probs = model.predict_proba(X_test)[:,1]
    fpr, tpr, _ = roc_curve(y_test, probs)
    roc_auc = auc(fpr, tpr)
    plt.plot(
        fpr,
        tpr,
        label=f"{nome} AUC={roc_auc:.3f}"
    )
plt.plot([0,1],[0,1],'--')
plt.xlabel("FPR")
plt.ylabel("TPR")
# plt.title("Comparação ROC")
plt.legend()
plt.savefig(f"{OUTPUT_DIR}/roc_comparativa.png")
plt.close()

print("Fim avaliação / validação modelos ")
```

```
Inciando Validação dos modelos


Top 20 features: Random Forest
            feature  importance
10            AAC_L    0.068468
250        DIPEP_NL    0.043401
1             AAC_A    0.028735
204        DIPEP_LE    0.026928
216        DIPEP_LS    0.021979
190        DIPEP_KL    0.020281
209        DIPEP_LK    0.016903
330        DIPEP_SL    0.016161
214        DIPEP_LQ    0.014612
433  NUM_NLR_MOTIFS    0.013699
70         DIPEP_DL    0.013470
```

```
21          DIPEP_AA    0.013068
0     protein_length    0.012937
203         DIPEP_LD    0.012824
150         DIPEP_HL    0.012078
215         DIPEP_LR    0.012028
90          DIPEP_EL    0.011851
217         DIPEP_LT    0.011333
121         DIPEP_GA    0.006368
6              AAC_G    0.006210

Top 20 features: XGBoost
                      feature  importance
10                      AAC_L    0.047183
438  dist_ploop_kinase2_norm    0.025910
1                       AAC_A    0.017776
441                 has_ploop    0.012968
422                   KINASE2    0.012358
434    num_motifs_detectados    0.011474
250                 DIPEP_NL    0.010831
222                 DIPEP_MC    0.009954
0             protein_length    0.008961
433           NUM_NLR_MOTIFS    0.007998
437            dist_glpl_mhd    0.007411
429                      HRD    0.006833
139                 DIPEP_GW    0.006488
146                 DIPEP_HG    0.005867
205                 DIPEP_LF    0.005848
6                      AAC_G    0.005416
239                 DIPEP_MW    0.005201
4                      AAC_E    0.005072
322                 DIPEP_SC    0.004825
204                 DIPEP_LE    0.004726

COMPARAÇÃO MODELOS
           Modelo   ROC_AUC        F1  Precision    Recall  Balanced_Accuracy  \
0  Random Forest  0.942413  0.861472   0.872807  0.850427           0.863115
1        XGBoost  0.950214  0.879023   0.914550  0.846154           0.883462
2            SVM  0.945442  0.873269   0.870488  0.876068           0.872724

   Accuracy       MCC
0  0.863102  0.726450
1  0.883422  0.769021
2  0.872727  0.745468
Fim avaliação / validação modelos
```

[13]:
```
'''
Funções para rodar a aplicação
```

```
'''

df_validacao = []

def validar_motifs(df,planta,modelo):
    print("\nVALIDAÇÃO BIOLÓGICA\n")
    counts = {}
    for motif_name, pattern in motifs.items():
        counts[motif_name] = 0
        for seq in df["protein_seq"]:
            if re.search(pattern, seq):
                counts[motif_name] += 1
    for k, v in counts.items():
        print(k, ":", v)

    counts['planta'] = planta
    counts['modelo'] = modelo

    df_validacao.append(counts)
    df_counts = pd.DataFrame([counts])
    df_counts.to_csv(
        f"{OUTPUT_DIR}/validacao_biologica_{modelo}_{planta}.csv",
        index=False
    )



def rodar_transdecoder(fasta_nt):
    cmd1 = [
        "TransDecoder.LongOrfs",
        "-t",
        fasta_nt,
        "--output_dir",
        f"{OUTPUT_DIR}"
    ]
    with open(fasta_nt+"_output1.txt", "a", encoding="utf-8") as log_file:
        subprocess.run(cmd1,
                       stdout=log_file,
                       stderr=subprocess.STDOUT,
                       text=True)

    cmd2 = [
        "TransDecoder.Predict",
        "-t",
        fasta_nt,
        "--output_dir",
        f"{OUTPUT_DIR}"
```

```
    ]
    with open(fasta_nt+"_output2.txt", "a", encoding="utf-8") as log_file:
        subprocess.run(cmd2,
                       stdout=log_file,
                       stderr=subprocess.STDOUT,
                       text=True)
    pep_file = fasta_nt + ".transdecoder.pep"
    return pep_file


def predizer_transcritos(fasta_nt,model,nmodel, output, planta):
    probabilidade = 0.85
    candidatos = []
    pep_file = rodar_transdecoder(fasta_nt)
    for record in SeqIO.parse(fasta_nt+".transdecoder.pep", "fasta"):
        protein = str(record.seq)
        feats = extrair_features_proteina(protein)
        if feats is None:
            continue
        X = scaler.transform([feats])
        prob = model.predict_proba(X)[0][1]
        candidatos.append({
            "id": record.id,
            "descricao": record.description,
            "protein_length": len(protein),
            "probabilidade_resistencia": prob,
            "protein_seq": protein
        })

    candidatos_df = pd.DataFrame(candidatos)
    candidatos_df = candidatos_df.sort_values(
        "probabilidade_resistencia",
        ascending=False
    )

    # top = candidatos_df.head(40)
    top = candidatos_df[candidatos_df["probabilidade_resistencia"] >=␣
 ↪probabilidade]
    print("\nTOP CANDIDATOS com probabilidade de >= ")
    print(probabilidade)
    print(fasta_nt+", MODELO: "+nmodel)
    print(top[[
        "id",
        "descricao",
        "protein_length",
        "probabilidade_resistencia"
    ]])
```

```
    validar_motifs(top,planta,nmodel)

    candidatos_df.to_csv(
        f"{OUTPUT_DIR}/{output}",
        index=False
    )
```

[14]:
```
'''
Aplicação - Modelo XGBoost
'''
print("\n\n=================Inciando Aplicação: Predição de␣
 ↪Genes===============\n\n")
print("\n\n Predição Modelo XGBoost")
inicio = datetime.now()
print("\n\n\nInciando predição ARABIDOPSIS: ", inicio.strftime("%H:%M:%S"))

print("\nPredição para ARABIDOPSIS\n")
predizer_transcritos(transcriptoma_arabidopsis_fasta,␣
 ↪xgb_model,'XGBoost',output="predicoes_arabidopsis_xgboost.
 ↪csv",planta="Arabidopsis thaliana")

fim = datetime.now()
print("\n\nFinalizado predição ARABIDOPSIS:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)
```

```
=================Inciando Aplicação: Predição de Genes===============



 Predição Modelo XGBoost



Inciando predição ARABIDOPSIS:  17:42:01

Predição para ARABIDOPSIS


TOP CANDIDATOS com probabilidade de >=
0.85
pipelineAD/arabidopsis_transcriptoma.fasta, MODELO: XGBoost
                      id                                          descricao  \
7120      NM_118991.3.p1  NM_118991.3.p1 GENE.NM_118991.3~~NM_118991.3.p…
5478      NM_114827.4.p1  NM_114827.4.p1 GENE.NM_114827.4~~NM_114827.4.p…
```

```
10167     NM_125967.5.p1  NM_125967.5.p1 GENE.NM_125967.5~~NM_125967.5.p…
2027   NM_001341467.1.p1  NM_001341467.1.p1 GENE.NM_001341467.1~~NM_0013…
4520   NM_001344672.1.p1  NM_001344672.1.p1 GENE.NM_001344672.1~~NM_0013…
…                    …                                                …
6919      NM_118549.2.p1  NM_118549.2.p1 GENE.NM_118549.2~~NM_118549.2.p…
4528   NM_001344681.1.p1  NM_001344681.1.p1 GENE.NM_001344681.1~~NM_0013…
1291   NM_001340444.1.p1  NM_001340444.1.p1 GENE.NM_001340444.1~~NM_0013…
4784   NM_001345025.1.p1  NM_001345025.1.p1 GENE.NM_001345025.1~~NM_0013…
4785   NM_001345026.1.p1  NM_001345026.1.p1 GENE.NM_001345026.1~~NM_0013…

       protein_length  probabilidade_resistencia
7120             1000                   0.999992
5478             1003                   0.999984
10167            1004                   0.999983
2027             1038                   0.999979
4520             1174                   0.999978
…                 …                      …
6919              876                   0.854744
4528              601                   0.853979
1291              426                   0.851583
4784              722                   0.850072
4785              722                   0.850072

[764 rows x 4 columns]

VALIDAÇÃO BIOLÓGICA

P_LOOP : 290
KINASE2 : 34
GLPL : 399
MHD : 418
RNBS_A : 0
RNBS_B : 525
RNBS_C : 608
RNBS_D : 664
HRD : 73
DFG : 152
LRR1 : 108
LRR2 : 228


Finalizado predição ARABIDOPSIS: 17:44:04

Tempo decorrido: 0:02:03.416514
```

[15]:
```
print("\nPredição para MELONGENA\n")
inicio = datetime.now()
```

```
print("\n\n\nInciando predição MELONGENA: ", inicio.strftime("%H:%M:%S"))
predizer_transcritos(transcriptoma_melongena_fasta,␣
 ↪xgb_model,'XGBoost',output="predicoes_melongena_xgboost.csv",planta="Solanum␣
 ↪melongena")
fim = datetime.now()
print("\n\nFinalizado predição MELONGENA:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)
```

```
Predição para MELONGENA



Inciando predição MELONGENA:  17:44:04

TOP CANDIDATOS com probabilidade de >=
0.85
pipelineAD/melongena_transcriptoma.fasta, MODELO: XGBoost
                     id                                                  descricao  \
5654  GBEF01028998.1.p1  GBEF01028998.1.p1 GENE.GBEF01028998.1~~GBEF010…
4652  GBEF01026373.1.p1  GBEF01026373.1.p1 GENE.GBEF01026373.1~~GBEF010…
4750  GBEF01026657.1.p1  GBEF01026657.1.p1 GENE.GBEF01026657.1~~GBEF010…
8768  GBEF01043405.1.p1  GBEF01043405.1.p1 GENE.GBEF01043405.1~~GBEF010…
8769  GBEF01043406.1.p1  GBEF01043406.1.p1 GENE.GBEF01043406.1~~GBEF010…
…                   …                                                     …
8664  GBEF01043181.1.p1  GBEF01043181.1.p1 GENE.GBEF01043181.1~~GBEF010…
6164  GBEF01031074.1.p2  GBEF01031074.1.p2 GENE.GBEF01031074.1~~GBEF010…
4978  GBEF01027267.1.p1  GBEF01027267.1.p1 GENE.GBEF01027267.1~~GBEF010…
5539  GBEF01028649.1.p3  GBEF01028649.1.p3 GENE.GBEF01028649.1~~GBEF010…
5542  GBEF01028650.1.p3  GBEF01028650.1.p3 GENE.GBEF01028650.1~~GBEF010…

      protein_length  probabilidade_resistencia
5654            1005                   0.999983
4652             671                   0.999983
4750             815                   0.999982
8768             921                   0.999978
8769             915                   0.999975
…                …                          …
8664             602                   0.853471
6164             176                   0.852417
4978             819                   0.851956
5539             111                   0.850376
5542             111                   0.850376

[311 rows x 4 columns]

VALIDAÇÃO BIOLÓGICA
```

```
P_LOOP : 36
KINASE2 : 12
GLPL : 123
MHD : 135
RNBS_A : 0
RNBS_B : 158
RNBS_C : 214
RNBS_D : 256
HRD : 44
DFG : 56
LRR1 : 31
LRR2 : 63


Finalizado predição MELONGENA: 17:45:50

Tempo decorrido: 0:01:45.539350
```

[16]:
```
print("\nPredição para TUBEROSUM\n")
inicio = datetime.now()
print("\n\n\nInciando predição TUBEROSUM: ", inicio.strftime("%H:%M:%S"))
predizer_transcritos(transcriptoma_tuberosum_fasta,␣
  ↪xgb_model,'XGBoost',output="predicoes_tuberosum_xgboost.csv",planta="Solanum␣
  ↪tuberosum")
fim = datetime.now()
print("\n\nFinalizado predição TUBEROSUM:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)
```

```
Predição para TUBEROSUM




Inciando predição TUBEROSUM:  17:45:50

TOP CANDIDATOS com probabilidade de >=
0.85
pipelineAD/tuberosum_transcriptoma.fasta, MODELO: XGBoost
                        id                                                descricao  \
2995   XM_006358666.2.p1  XM_006358666.2.p1 GENE.XM_006358666.2~~XM_0063…
9951   XM_015314054.1.p1  XM_015314054.1.p1 GENE.XM_015314054.1~~XM_0153…
10158  XM_015314468.1.p1  XM_015314468.1.p1 GENE.XM_015314468.1~~XM_0153…
3361   XM_006359509.2.p1  XM_006359509.2.p1 GENE.XM_006359509.2~~XM_0063…
7408   XM_015304012.1.p1  XM_015304012.1.p1 GENE.XM_015304012.1~~XM_0153…
…                    …                                                 …
10558  XM_015315339.1.p1  XM_015315339.1.p1 GENE.XM_015315339.1~~XM_0153…
```

```
3650   XM_006360176.2.p1  XM_006360176.2.p1 GENE.XM_006360176.2~~XM_0063…
3297   XM_006359359.2.p1  XM_006359359.2.p1 GENE.XM_006359359.2~~XM_0063…
385    XM_006352927.2.p1  XM_006352927.2.p1 GENE.XM_006352927.2~~XM_0063…
8419   XM_015311148.1.p1  XM_015311148.1.p1 GENE.XM_015311148.1~~XM_0153…

       protein_length  probabilidade_resistencia
2995             1053                   0.999994
9951             1030                   0.999994
10158            1162                   0.999993
3361             1160                   0.999993
7408             3960                   0.999992
…                …                       …
10558            1863                   0.851779
3650              498                   0.850933
3297              630                   0.850400
385              1741                   0.850235
8419             1741                   0.850235

[1069 rows x 4 columns]

VALIDAÇÃO BIOLÓGICA

P_LOOP : 398
KINASE2 : 174
GLPL : 582
MHD : 623
RNBS_A : 0
RNBS_B : 701
RNBS_C : 859
RNBS_D : 998
HRD : 124
DFG : 244
LRR1 : 176
LRR2 : 340


Finalizado predição TUBEROSUM: 17:47:56

Tempo decorrido: 0:02:06.193502
```

[17]:
```
print("\nPredição para PHYSALIS\n")
inicio = datetime.now()
print("\n\n\nInciando predição PHYSALIS: ", inicio.strftime("%H:%M:%S"))
predizer_transcritos(transcriptoma_physalis_fasta,␣
 ↪xgb_model,'XGBoost',output="predicoes_physalis_xgboost.csv",planta="Physalis␣
 ↪peruviana")
fim = datetime.now()
```

```
print("\n\nFinalizado predição TUBEROSUM:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)
print("\nArquivo salvo:")
print(f"{OUTPUT_DIR}/predicoes_xgboost.csv")
```

```
Predição para PHYSALIS




Inciando predição PHYSALIS:  17:47:56

TOP CANDIDATOS com probabilidade de >=
0.85
pipelineAD/physalis_transcriptoma.fasta, MODELO: XGBoost
                        id                                                  descricao  \
7025   IABH01018591.1.p1  IABH01018591.1.p1 GENE.IABH01018591.1~~IABH010…
7026   IABH01018592.1.p1  IABH01018592.1.p1 GENE.IABH01018592.1~~IABH010…
6132   IABH01016994.1.p1  IABH01016994.1.p1 GENE.IABH01016994.1~~IABH010…
11354  IABH01035124.1.p1  IABH01035124.1.p1 GENE.IABH01035124.1~~IABH010…
11274  IABH01034764.1.p1  IABH01034764.1.p1 GENE.IABH01034764.1~~IABH010…
…                    …                                                   …
3539   IABH01011574.1.p1  IABH01011574.1.p1 GENE.IABH01011574.1~~IABH010…
3412   IABH01011278.1.p1  IABH01011278.1.p1 GENE.IABH01011278.1~~IABH010…
6941   IABH01018459.1.p2  IABH01018459.1.p2 GENE.IABH01018459.1~~IABH010…
4147   IABH01013331.1.p1  IABH01013331.1.p1 GENE.IABH01013331.1~~IABH010…
4581   IABH01014128.1.p1  IABH01014128.1.p1 GENE.IABH01014128.1~~IABH010…

       protein_length  probabilidade_resistencia
7025             1081                   0.999995
7026             1081                   0.999995
6132             1219                   0.999992
11354            1233                   0.999992
11274            1014                   0.999988
…                   …                        …
3539              807                   0.859725
3412              602                   0.859594
6941              113                   0.859403
4147              693                   0.854953
4581              871                   0.852411

[538 rows x 4 columns]

VALIDAÇÃO BIOLÓGICA

P_LOOP : 120
KINASE2 : 31
```

```
GLPL : 246
MHD : 279
RNBS_A : 0
RNBS_B : 322
RNBS_C : 407
RNBS_D : 473
HRD : 67
DFG : 99
LRR1 : 67
LRR2 : 147


Finalizado predição TUBEROSUM: 17:49:55

Tempo decorrido: 0:01:59.561325

Arquivo salvo:
pipelineAD/predicoes_xgboost.csv
```

[18]:
```
print(df_validacao)
```

```
[{'P_LOOP': 290, 'KINASE2': 34, 'GLPL': 399, 'MHD': 418, 'RNBS_A': 0, 'RNBS_B':
525, 'RNBS_C': 608, 'RNBS_D': 664, 'HRD': 73, 'DFG': 152, 'LRR1': 108, 'LRR2':
228, 'planta': 'Arabidopsis thaliana', 'modelo': 'XGBoost'}, {'P_LOOP': 36,
'KINASE2': 12, 'GLPL': 123, 'MHD': 135, 'RNBS_A': 0, 'RNBS_B': 158, 'RNBS_C':
214, 'RNBS_D': 256, 'HRD': 44, 'DFG': 56, 'LRR1': 31, 'LRR2': 63, 'planta':
'Solanum melongena', 'modelo': 'XGBoost'}, {'P_LOOP': 398, 'KINASE2': 174,
'GLPL': 582, 'MHD': 623, 'RNBS_A': 0, 'RNBS_B': 701, 'RNBS_C': 859, 'RNBS_D':
998, 'HRD': 124, 'DFG': 244, 'LRR1': 176, 'LRR2': 340, 'planta': 'Solanum
tuberosum', 'modelo': 'XGBoost'}, {'P_LOOP': 120, 'KINASE2': 31, 'GLPL': 246,
'MHD': 279, 'RNBS_A': 0, 'RNBS_B': 322, 'RNBS_C': 407, 'RNBS_D': 473, 'HRD': 67,
'DFG': 99, 'LRR1': 67, 'LRR2': 147, 'planta': 'Physalis peruviana', 'modelo':
'XGBoost'}]
```

[19]:
```
'''
Aplicação - Modelo Random Forest
'''
print("\n\n Predição Modelo RF")
print("\nPredição para ARABIDOPSIS\n")
inicio = datetime.now()
print("\n\n\nInciando predição ARABIDOPSIS: ", inicio.strftime("%H:%M:%S"))
predizer_transcritos(transcriptoma_arabidopsis_fasta,␣
 ↪rf_model,"RF",output="predicoes_arabidopsis_rf.csv",planta="Arabidopsis␣
 ↪thaliana")
fim = datetime.now()
print("\n\nFinalizado predição ARABIDOPSIS:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)
```

```
 Predição Modelo RF

Predição para ARABIDOPSIS



Inciando predição ARABIDOPSIS:  17:49:55

TOP CANDIDATOS com probabilidade de >=
0.85
pipelineAD/arabidopsis_transcriptoma.fasta, MODELO: RF
                       id                                              descricao  \
3611   NM_001343517.1.p1  NM_001343517.1.p1 GENE.NM_001343517.1~~NM_0013…
6758      NM_118133.3.p1  NM_118133.3.p1 GENE.NM_118133.3~~NM_118133.3.p…
7120      NM_118991.3.p1  NM_118991.3.p1 GENE.NM_118991.3~~NM_118991.3.p…
8930      NM_123051.4.p1  NM_123051.4.p1 GENE.NM_123051.4~~NM_123051.4.p…
4122   NM_001344160.1.p1  NM_001344160.1.p1 GENE.NM_001344160.1~~NM_0013…
…                    …                                                 …
4443   NM_001344582.1.p1  NM_001344582.1.p1 GENE.NM_001344582.1~~NM_0013…
9085      NM_123519.2.p1  NM_123519.2.p1 GENE.NM_123519.2~~NM_123519.2.p…
2428   NM_001341998.1.p1  NM_001341998.1.p1 GENE.NM_001341998.1~~NM_0013…
103    NM_001036894.3.p1  NM_001036894.3.p1 GENE.NM_001036894.3~~NM_0010…
10292     NM_148048.3.p1  NM_148048.3.p1 GENE.NM_148048.3~~NM_148048.3.p…

       protein_length  probabilidade_resistencia
3611             1296                   0.991718
6758             1250                   0.989641
7120             1000                   0.986981
8930             1189                   0.985729
4122             1189                   0.985729
…                …                       …
4443             1213                   0.853919
9085             1086                   0.853262
2428             1970                   0.853160
103               456                   0.850992
10292             456                   0.850992

[269 rows x 4 columns]

VALIDAÇÃO BIOLÓGICA

P_LOOP : 163
KINASE2 : 26
GLPL : 203
MHD : 179
RNBS_A : 0
```

```
RNBS_B : 249
RNBS_C : 256
RNBS_D : 263
HRD : 45
DFG : 94
LRR1 : 67
LRR2 : 136


Finalizado predição ARABIDOPSIS: 18:03:38

Tempo decorrido: 0:13:42.220287
```

[20]:
```
print("\nPredição para MELONGENA\n")
inicio = datetime.now()
print("\n\n\nInciando predição MELONGENA: ", inicio.strftime("%H:%M:%S"))
predizer_transcritos(transcriptoma_melongena_fasta,␣
 ↪rf_model,'RF',output="predicoes_melongena_rf.csv",planta="Solanum melongena")
fim = datetime.now()
print("\n\nFinalizado predição MELONGENA:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)
```

```
Predição para MELONGENA



Inciando predição MELONGENA:  18:03:38

TOP CANDIDATOS com probabilidade de >=
0.85
pipelineAD/melongena_transcriptoma.fasta, MODELO: RF
                     id                                                 descricao  \
8768  GBEF01043405.1.p1  GBEF01043405.1.p1 GENE.GBEF01043405.1~~GBEF010…
8769  GBEF01043406.1.p1  GBEF01043406.1.p1 GENE.GBEF01043406.1~~GBEF010…
8179  GBEF01041955.1.p1  GBEF01041955.1.p1 GENE.GBEF01041955.1~~GBEF010…
4134  GBEF01025346.1.p1  GBEF01025346.1.p1 GENE.GBEF01025346.1~~GBEF010…
404   GBEF01001526.1.p1  GBEF01001526.1.p1 GENE.GBEF01001526.1~~GBEF010…
…                   …                                                  …
3223  GBEF01023316.1.p1  GBEF01023316.1.p1 GENE.GBEF01023316.1~~GBEF010…
4489  GBEF01026077.1.p1  GBEF01026077.1.p1 GENE.GBEF01026077.1~~GBEF010…
3222  GBEF01023315.1.p1  GBEF01023315.1.p1 GENE.GBEF01023315.1~~GBEF010…
9241  GBEF01044460.1.p1  GBEF01044460.1.p1 GENE.GBEF01044460.1~~GBEF010…
9355      MZ826511.1.p1  MZ826511.1.p1 GENE.MZ826511.1~~MZ826511.1.p1  …

      protein_length  probabilidade_resistencia
8768             921                   0.993342
```

```
8769              915                   0.990327
8179              959                   0.981195
4134             1022                   0.980369
404               559                   0.977285
…                 …                      …
3223              874                   0.866996
4489              866                   0.864688
3222              883                   0.859933
9241              454                   0.856594
9355              590                   0.851450

[67 rows x 4 columns]

VALIDAÇÃO BIOLÓGICA

P_LOOP : 14
KINASE2 : 10
GLPL : 45
MHD : 22
RNBS_A : 0
RNBS_B : 40
RNBS_C : 53
RNBS_D : 66
HRD : 17
DFG : 19
LRR1 : 11
LRR2 : 19


Finalizado predição MELONGENA: 18:15:49

Tempo decorrido: 0:12:11.534931
```

[21]:
```
print("\nPredição para TUBEROSUM\n")
inicio = datetime.now()
print("\n\n\nInciando predição TUBEROSUM: ", inicio.strftime("%H:%M:%S"))
predizer_transcritos(transcriptoma_tuberosum_fasta,␣
  ↪rf_model,"RF",output="predicoes_tuberosum_rf.csv",planta="Solanum tuberosum")
fim = datetime.now()
print("\n\nFinalizado predição TUBEROSUM:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)
```

```
Predição para TUBEROSUM
```

```
Inciando predição TUBEROSUM:  18:15:49

TOP CANDIDATOS com probabilidade de >=
0.85
pipelineAD/tuberosum_transcriptoma.fasta, MODELO: RF
                       id                                                descricao  \
4253   XM_006361689.2.p1  XM_006361689.2.p1 GENE.XM_006361689.2~~XM_0063…
931    XM_006354074.2.p1  XM_006354074.2.p1 GENE.XM_006354074.2~~XM_0063…
9752   XM_015313795.1.p1  XM_015313795.1.p1 GENE.XM_015313795.1~~XM_0153…
2726   XM_006358087.2.p1  XM_006358087.2.p1 GENE.XM_006358087.2~~XM_0063…
6576   XM_006367078.2.p1  XM_006367078.2.p1 GENE.XM_006367078.2~~XM_0063…
…                    …                                                  …
2005   XM_006356430.2.p1  XM_006356430.2.p1 GENE.XM_006356430.2~~XM_0063…
9263   XM_015312932.1.p1  XM_015312932.1.p1 GENE.XM_015312932.1~~XM_0153…
141    XM_006352363.2.p1  XM_006352363.2.p1 GENE.XM_006352363.2~~XM_0063…
10454  XM_015315069.1.p1  XM_015315069.1.p1 GENE.XM_015315069.1~~XM_0153…
9731   XM_015313767.1.p1  XM_015313767.1.p1 GENE.XM_015313767.1~~XM_0153…

       protein_length  probabilidade_resistencia
4253             1271                   0.999258
931               863                   0.998589
9752             1176                   0.998141
2726             1176                   0.998141
6576             1092                   0.997912
…                   …                          …
2005              587                   0.853067
9263              575                   0.853033
141               441                   0.852432
10454             462                   0.851199
9731              996                   0.851047

[535 rows x 4 columns]

VALIDAÇÃO BIOLÓGICA

P_LOOP : 270
KINASE2 : 143
GLPL : 364
MHD : 369
RNBS_A : 0
RNBS_B : 401
RNBS_C : 480
RNBS_D : 533
HRD : 74
DFG : 170
LRR1 : 140
LRR2 : 274
```

```
Finalizado predição TUBEROSUM: 18:29:41

Tempo decorrido: 0:13:51.642922
```

[22]:
```
print("\nPredição para PHYSALIS\n")
inicio = datetime.now()
print("\n\n\nInciando predição PHYSALIS: ", inicio.strftime("%H:%M:%S"))
predizer_transcritos(transcriptoma_physalis_fasta,␣
 ↪rf_model,"RF",output="predicoes_physalis_rf.csv",planta="Physalis peruviana")
fim = datetime.now()
print("\n\nFinalizado predição PHYSALIS:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)

print("\nArquivo salvo:")
print(f"{OUTPUT_DIR}/predicoes_rf.csv")
```

```
Predição para PHYSALIS



Inciando predição PHYSALIS:  18:29:41

TOP CANDIDATOS com probabilidade de >=
0.85
pipelineAD/physalis_transcriptoma.fasta, MODELO: RF
                        id                                                 descricao  \
1674   IABH01006408.1.p1  IABH01006408.1.p1 GENE.IABH01006408.1~~IABH010…
7026   IABH01018592.1.p1  IABH01018592.1.p1 GENE.IABH01018592.1~~IABH010…
7025   IABH01018591.1.p1  IABH01018591.1.p1 GENE.IABH01018591.1~~IABH010…
6132   IABH01016994.1.p1  IABH01016994.1.p1 GENE.IABH01016994.1~~IABH010…
11354  IABH01035124.1.p1  IABH01035124.1.p1 GENE.IABH01035124.1~~IABH010…
…                    …                                                  …
3814   IABH01012318.1.p2  IABH01012318.1.p2 GENE.IABH01012318.1~~IABH010…
3812   IABH01012317.1.p2  IABH01012317.1.p2 GENE.IABH01012317.1~~IABH010…
8313   IABH01020690.1.p1  IABH01020690.1.p1 GENE.IABH01020690.1~~IABH010…
186    IABH01000643.1.p1  IABH01000643.1.p1 GENE.IABH01000643.1~~IABH010…
4095   IABH01013179.1.p1  IABH01013179.1.p1 GENE.IABH01013179.1~~IABH010…

       protein_length  probabilidade_resistencia
1674              915                   0.998028
7026             1081                   0.997511
7025             1081                   0.997511
6132             1219                   0.996281
11354            1233                   0.992644
…                   …                          …
```

```
3814              269                 0.855415
3812              269                 0.855415
8313              357                 0.855193
186               598                 0.854675
4095              851                 0.854119

[165 rows x 4 columns]

VALIDAÇÃO BIOLÓGICA

P_LOOP : 48
KINASE2 : 27
GLPL : 100
MHD : 98
RNBS_A : 0
RNBS_B : 122
RNBS_C : 146
RNBS_D : 165
HRD : 28
DFG : 43
LRR1 : 31
LRR2 : 73


Finalizado predição PHYSALIS: 18:44:30

Tempo decorrido: 0:14:49.490441

Arquivo salvo:
pipelineAD/predicoes_rf.csv
```

[23]:
```
'''
Aplicação - Modelo SVM
'''
print("\n\n Predição Modelo SVM")
print("\nPredição para ARABIDOPSIS\n")
inicio = datetime.now()
print("\n\n\nInciando predição ARABIDOPSIS: ", inicio.strftime("%H:%M:%S"))
predizer_transcritos(transcriptoma_arabidopsis_fasta,␣
 ↪svm_model,"SVM",output="predicoes_arabidopsis_svm.csv",planta="Arabidopsis␣
 ↪thaliana")
fim = datetime.now()
print("\n\nFinalizado predição ARABIDOPSIS:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)
```

```
 Predição Modelo SVM
```

```
Predição para ARABIDOPSIS



Inciando predição ARABIDOPSIS:  18:44:30

TOP CANDIDATOS com probabilidade de >=
0.85
pipelineAD/arabidopsis_transcriptoma.fasta, MODELO: SVM
                       id                                              descricao  \
6231      NM_116660.3.p1  NM_116660.3.p1 GENE.NM_116660.3~~NM_116660.3.p…
688    NM_001339603.1.p1  NM_001339603.1.p1 GENE.NM_001339603.1~~NM_0013…
5632      NM_115184.5.p1  NM_115184.5.p1 GENE.NM_115184.5~~NM_115184.5.p…
10002     NM_125617.3.p1  NM_125617.3.p1 GENE.NM_125617.3~~NM_125617.3.p…
8680      NM_122246.3.p1  NM_122246.3.p1 GENE.NM_122246.3~~NM_122246.3.p…
…                    …                                                 …
8951      NM_123113.3.p1  NM_123113.3.p1 GENE.NM_123113.3~~NM_123113.3.p…
1020   NM_001340084.1.p1  NM_001340084.1.p1 GENE.NM_001340084.1~~NM_0013…
2409   NM_001341978.1.p1  NM_001341978.1.p1 GENE.NM_001341978.1~~NM_0013…
6490      NM_117486.4.p1  NM_117486.4.p1 GENE.NM_117486.4~~NM_117486.4.p…
1612   NM_001340900.1.p1  NM_001340900.1.p1 GENE.NM_001340900.1~~NM_0013…

       protein_length  probabilidade_resistencia
6231              812                   1.000000
688               892                   1.000000
5632              892                   1.000000
10002             967                   1.000000
8680              590                   1.000000
…                 …                      …
8951              443                   0.852601
1020              472                   0.852211
2409              819                   0.851072
6490              469                   0.850160
1612              469                   0.850160

[650 rows x 4 columns]

VALIDAÇÃO BIOLÓGICA

P_LOOP : 268
KINASE2 : 29
GLPL : 330
MHD : 348
RNBS_A : 0
RNBS_B : 472
RNBS_C : 542
```

```
RNBS_D : 589
HRD : 73
DFG : 143
LRR1 : 106
LRR2 : 197


Finalizado predição ARABIDOPSIS: 18:44:37

Tempo decorrido: 0:00:06.302126
```

[24]:
```
print("\nPredição para MELONGENA\n")
inicio = datetime.now()
print("\n\n\nInciando predição MELONGENA: ", inicio.strftime("%H:%M:%S"))
predizer_transcritos(transcriptoma_melongena_fasta,␣
 ↪svm_model,'SVM',output="predicoes_melongena_svm.csv",planta="Solanum␣
 ↪melongena")
fim = datetime.now()
print("\n\nFinalizado predição MELONGENA:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)
```

```
Predição para MELONGENA



Inciando predição MELONGENA:  18:44:37

TOP CANDIDATOS com probabilidade de >=
0.85
pipelineAD/melongena_transcriptoma.fasta, MODELO: SVM
                     id                                                 descricao  \
8768  GBEF01043405.1.p1  GBEF01043405.1.p1 GENE.GBEF01043405.1~~GBEF010…
8769  GBEF01043406.1.p1  GBEF01043406.1.p1 GENE.GBEF01043406.1~~GBEF010…
404   GBEF01001526.1.p1  GBEF01001526.1.p1 GENE.GBEF01001526.1~~GBEF010…
4652  GBEF01026373.1.p1  GBEF01026373.1.p1 GENE.GBEF01026373.1~~GBEF010…
4134  GBEF01025346.1.p1  GBEF01025346.1.p1 GENE.GBEF01025346.1~~GBEF010…
…                   …                                                  …
3952  GBEF01024963.1.p1  GBEF01024963.1.p1 GENE.GBEF01024963.1~~GBEF010…
9250  GBEF01044472.1.p1  GBEF01044472.1.p1 GENE.GBEF01044472.1~~GBEF010…
7022  GBEF01034427.1.p1  GBEF01034427.1.p1 GENE.GBEF01034427.1~~GBEF010…
5287  GBEF01028088.1.p1  GBEF01028088.1.p1 GENE.GBEF01028088.1~~GBEF010…
3677  GBEF01024343.1.p1  GBEF01024343.1.p1 GENE.GBEF01024343.1~~GBEF010…

      protein_length  probabilidade_resistencia
8768             921                   1.000000
8769             915                   1.000000
```

```
404              559                   1.000000
4652             671                   1.000000
4134            1022                   1.000000
…                …                      …
3952            1163                   0.855324
9250             563                   0.851193
7022            1128                   0.850444
5287             486                   0.850432
3677             655                   0.850310

[339 rows x 4 columns]

VALIDAÇÃO BIOLÓGICA

P_LOOP : 60
KINASE2 : 12
GLPL : 131
MHD : 148
RNBS_A : 0
RNBS_B : 200
RNBS_C : 229
RNBS_D : 291
HRD : 49
DFG : 65
LRR1 : 36
LRR2 : 62


Finalizado predição MELONGENA: 18:44:42

Tempo decorrido: 0:00:05.370915
```

[25]:
```
print("\nPredição para TUBEROSUM\n")
inicio = datetime.now()
print("\n\n\nInciando predição TUBEROSUM: ", inicio.strftime("%H:%M:%S"))
predizer_transcritos(transcriptoma_tuberosum_fasta,␣
 ↪svm_model,"SVM",output="predicoes_tuberosum_svm.csv",planta="Solanum␣
 ↪tuberosum")
fim = datetime.now()
print("\n\nFinalizado predição TUBEROSUM:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)
```

```
Predição para TUBEROSUM
```

```
Inciando predição TUBEROSUM:  18:44:42

TOP CANDIDATOS com probabilidade de >=
0.85
pipelineAD/tuberosum_transcriptoma.fasta, MODELO: SVM
                     id                                                descricao  \
6971  XM_015302997.1.p1  XM_015302997.1.p1 GENE.XM_015302997.1~~XM_0153…
4556  XM_006362392.2.p1  XM_006362392.2.p1 GENE.XM_006362392.2~~XM_0063…
2851  XM_006358355.2.p1  XM_006358355.2.p1 GENE.XM_006358355.2~~XM_0063…
3303  XM_006359386.2.p1  XM_006359386.2.p1 GENE.XM_006359386.2~~XM_0063…
7404  XM_015304006.1.p1  XM_015304006.1.p1 GENE.XM_015304006.1~~XM_0153…
…                   …                                                 …
8378  XM_015311059.1.p1  XM_015311059.1.p1 GENE.XM_015311059.1~~XM_0153…
9096  XM_015312568.1.p1  XM_015312568.1.p1 GENE.XM_015312568.1~~XM_0153…
341   XM_006352852.2.p1  XM_006352852.2.p1 GENE.XM_006352852.2~~XM_0063…
3998  XM_006361088.2.p1  XM_006361088.2.p1 GENE.XM_006361088.2~~XM_0063…
3951  XM_006360979.2.p1  XM_006360979.2.p1 GENE.XM_006360979.2~~XM_0063…

      protein_length  probabilidade_resistencia
6971            1050                   1.000000
4556            1018                   1.000000
2851            1220                   1.000000
3303             815                   1.000000
7404             996                   1.000000
…                …                          …
8378            1055                   0.850911
9096             491                   0.850726
341              391                   0.850453
3998             495                   0.850296
3951            1070                   0.850264

[966 rows x 4 columns]

VALIDAÇÃO BIOLÓGICA

P_LOOP : 376
KINASE2 : 165
GLPL : 512
MHD : 592
RNBS_A : 0
RNBS_B : 688
RNBS_C : 785
RNBS_D : 908
HRD : 136
DFG : 254
LRR1 : 174
LRR2 : 329
```

```
Finalizado predição TUBEROSUM: 18:44:48

Tempo decorrido: 0:00:06.286429
```

[26]:
```
print("\nPredição para PHYSALIS\n")
inicio = datetime.now()
print("\n\n\nInciando predição PHYSALIS: ", inicio.strftime("%H:%M:%S"))
predizer_transcritos(transcriptoma_physalis_fasta,␣
 ↪svm_model,"SVM",output="predicoes_physalis_svm.csv",planta="Physalis␣
 ↪peruviana")
fim = datetime.now()
print("\n\nFinalizado predição PHYSALIS:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)

print("\nArquivo salvo:")
print(f"{OUTPUT_DIR}/predicoes_svm.csv")
```

```
Predição para PHYSALIS



Inciando predição PHYSALIS:  18:44:48

TOP CANDIDATOS com probabilidade de >=
0.85
pipelineAD/physalis_transcriptoma.fasta, MODELO: SVM
                     id                                                 descricao  \
6132  IABH01016994.1.p1  IABH01016994.1.p1 GENE.IABH01016994.1~~IABH010…
9678  IABH01023011.1.p1  IABH01023011.1.p1 GENE.IABH01023011.1~~IABH010…
1674  IABH01006408.1.p1  IABH01006408.1.p1 GENE.IABH01006408.1~~IABH010…
3098  IABH01010640.1.p1  IABH01010640.1.p1 GENE.IABH01010640.1~~IABH010…
9680  IABH01023012.1.p1  IABH01023012.1.p1 GENE.IABH01023012.1~~IABH010…
…                   …                                                  …
3351  IABH01011153.1.p1  IABH01011153.1.p1 GENE.IABH01011153.1~~IABH010…
2166  IABH01008303.1.p1  IABH01008303.1.p1 GENE.IABH01008303.1~~IABH010…
6017  IABH01016753.1.p1  IABH01016753.1.p1 GENE.IABH01016753.1~~IABH010…
4027  IABH01013013.1.p2  IABH01013013.1.p2 GENE.IABH01013013.1~~IABH010…
3312  IABH01011064.1.p1  IABH01011064.1.p1 GENE.IABH01011064.1~~IABH010…

      protein_length  probabilidade_resistencia
6132            1219                   1.000000
9678             651                   1.000000
1674             915                   1.000000
3098             958                   1.000000
9680             919                   1.000000
```

```
…              …                    …
3351           561                  0.851623
2166           857                  0.851463
6017           235                  0.851302
4027           480                  0.850473
3312           546                  0.850014

[512 rows x 4 columns]

VALIDAÇÃO BIOLÓGICA

P_LOOP : 123
KINASE2 : 31
GLPL : 223
MHD : 284
RNBS_A : 0
RNBS_B : 338
RNBS_C : 386
RNBS_D : 449
HRD : 75
DFG : 104
LRR1 : 66
LRR2 : 137


Finalizado predição PHYSALIS: 18:44:55

Tempo decorrido: 0:00:06.508841

Arquivo salvo:
pipelineAD/predicoes_svm.csv
```

[27]:
```
print(df_validacao)
```

```
[{'P_LOOP': 290, 'KINASE2': 34, 'GLPL': 399, 'MHD': 418, 'RNBS_A': 0, 'RNBS_B':
525, 'RNBS_C': 608, 'RNBS_D': 664, 'HRD': 73, 'DFG': 152, 'LRR1': 108, 'LRR2':
228, 'planta': 'Arabidopsis thaliana', 'modelo': 'XGBoost'}, {'P_LOOP': 36,
'KINASE2': 12, 'GLPL': 123, 'MHD': 135, 'RNBS_A': 0, 'RNBS_B': 158, 'RNBS_C':
214, 'RNBS_D': 256, 'HRD': 44, 'DFG': 56, 'LRR1': 31, 'LRR2': 63, 'planta':
'Solanum melongena', 'modelo': 'XGBoost'}, {'P_LOOP': 398, 'KINASE2': 174,
'GLPL': 582, 'MHD': 623, 'RNBS_A': 0, 'RNBS_B': 701, 'RNBS_C': 859, 'RNBS_D':
998, 'HRD': 124, 'DFG': 244, 'LRR1': 176, 'LRR2': 340, 'planta': 'Solanum
tuberosum', 'modelo': 'XGBoost'}, {'P_LOOP': 120, 'KINASE2': 31, 'GLPL': 246,
'MHD': 279, 'RNBS_A': 0, 'RNBS_B': 322, 'RNBS_C': 407, 'RNBS_D': 473, 'HRD': 67,
'DFG': 99, 'LRR1': 67, 'LRR2': 147, 'planta': 'Physalis peruviana', 'modelo':
'XGBoost'}, {'P_LOOP': 163, 'KINASE2': 26, 'GLPL': 203, 'MHD': 179, 'RNBS_A': 0,
'RNBS_B': 249, 'RNBS_C': 256, 'RNBS_D': 263, 'HRD': 45, 'DFG': 94, 'LRR1': 67,
'LRR2': 136, 'planta': 'Arabidopsis thaliana', 'modelo': 'RF'}, {'P_LOOP': 14,
```

```
'KINASE2': 10, 'GLPL': 45, 'MHD': 22, 'RNBS_A': 0, 'RNBS_B': 40, 'RNBS_C': 53,
'RNBS_D': 66, 'HRD': 17, 'DFG': 19, 'LRR1': 11, 'LRR2': 19, 'planta': 'Solanum
melongena', 'modelo': 'RF'}, {'P_LOOP': 270, 'KINASE2': 143, 'GLPL': 364, 'MHD':
369, 'RNBS_A': 0, 'RNBS_B': 401, 'RNBS_C': 480, 'RNBS_D': 533, 'HRD': 74, 'DFG':
170, 'LRR1': 140, 'LRR2': 274, 'planta': 'Solanum tuberosum', 'modelo': 'RF'},
{'P_LOOP': 48, 'KINASE2': 27, 'GLPL': 100, 'MHD': 98, 'RNBS_A': 0, 'RNBS_B':
122, 'RNBS_C': 146, 'RNBS_D': 165, 'HRD': 28, 'DFG': 43, 'LRR1': 31, 'LRR2': 73,
'planta': 'Physalis peruviana', 'modelo': 'RF'}, {'P_LOOP': 268, 'KINASE2': 29,
'GLPL': 330, 'MHD': 348, 'RNBS_A': 0, 'RNBS_B': 472, 'RNBS_C': 542, 'RNBS_D':
589, 'HRD': 73, 'DFG': 143, 'LRR1': 106, 'LRR2': 197, 'planta': 'Arabidopsis
thaliana', 'modelo': 'SVM'}, {'P_LOOP': 60, 'KINASE2': 12, 'GLPL': 131, 'MHD':
148, 'RNBS_A': 0, 'RNBS_B': 200, 'RNBS_C': 229, 'RNBS_D': 291, 'HRD': 49, 'DFG':
65, 'LRR1': 36, 'LRR2': 62, 'planta': 'Solanum melongena', 'modelo': 'SVM'},
{'P_LOOP': 376, 'KINASE2': 165, 'GLPL': 512, 'MHD': 592, 'RNBS_A': 0, 'RNBS_B':
688, 'RNBS_C': 785, 'RNBS_D': 908, 'HRD': 136, 'DFG': 254, 'LRR1': 174, 'LRR2':
329, 'planta': 'Solanum tuberosum', 'modelo': 'SVM'}, {'P_LOOP': 123, 'KINASE2':
31, 'GLPL': 223, 'MHD': 284, 'RNBS_A': 0, 'RNBS_B': 338, 'RNBS_C': 386,
'RNBS_D': 449, 'HRD': 75, 'DFG': 104, 'LRR1': 66, 'LRR2': 137, 'planta':
'Physalis peruviana', 'modelo': 'SVM'}]
```

[28]:
```
df_copia = df_validacao.copy()
df_inicial = pd.DataFrame(df_validacao)

df_validacao = df_inicial.melt(
    id_vars=["planta", "modelo"],
    var_name="motivo",
    value_name="contagem"
)
print(df_validacao.head())

df_validacao.to_csv(f"{OUTPUT_DIR}/df_validacao.csv", index=False)
```

```
                planta   modelo  motivo  contagem
0  Arabidopsis thaliana  XGBoost  P_LOOP       290
1     Solanum melongena  XGBoost  P_LOOP        36
2     Solanum tuberosum  XGBoost  P_LOOP       398
3    Physalis peruviana  XGBoost  P_LOOP       120
4  Arabidopsis thaliana       RF  P_LOOP       163
```

[29]:
```
#Scatterplot
plt.figure(figsize=(12,6))
sns.scatterplot(
    data=df_validacao,
    x="motivo",
    y="contagem",
    hue="planta",
    style="modelo",
    s=200
```

```
)
# plt.title( "Validação biológica dos motivos conservados")
plt.ylabel("Número de ocorrências")
plt.xlabel("Motivo")
plt.legend(
    bbox_to_anchor=(1.05,1),
    loc="upper left"
)
plt.tight_layout()
plt.savefig(f"{OUTPUT_DIR}/Validacao_biologica_dos_motivos_conservados.png")
plt.close()
# plt.show()

#Bubble Chart
plt.figure(figsize=(12,6))
sns.scatterplot(
    data=df_validacao,
    x="motivo",
    y="modelo",
    hue="planta",
    size="contagem",
    sizes=(50,600)
)
# plt.title("Distribuição dos motivos por modelo e espécie")
plt.tight_layout()
plt.savefig(f"{OUTPUT_DIR}/distribuicao_dos_motivos_por_modelo_especie.png")
plt.close()
# plt.show()


#Facetas por planta
g = sns.catplot(
    data=df_validacao,
    x="motivo",
    y="contagem",
    hue="modelo",
    col="planta",
    kind="bar",
    height=5,
    aspect=1.2
)
g.set_xticklabels(rotation=45)
plt.savefig(f"{OUTPUT_DIR}/facetas_por_planta.png")
plt.close()
# plt.show()

#por planta
```

```
#Physalis peruviana
df_physalis = df_validacao[df_validacao["planta"] == "Physalis peruviana"]
plt.figure(figsize=(12, 6))
sns.set_theme(style="whitegrid")
sns.lineplot(
    data=df_physalis,
    x="motivo",
    y="contagem",
    hue="modelo",
    style="modelo",
    markers=True,      # Ativa os marcadores nos pontos
    dashes=False,      # Mantém as linhas contínuas
    linewidth=2,       # Espessura da linha
    markersize=10      # Tamanho dos pontos de dispersão
)

# plt.title("Comparação dos Modelos para Physalis peruviana", fontsize=14,␣
 ↪fontweight="bold", pad=15)
plt.xlabel("Motivos Conservados", fontsize=12, labelpad=10)
plt.ylabel("Número de Ocorrências (Contagem)", fontsize=12, labelpad=10)
plt.xticks(rotation=45, ha="right")
plt.legend(title="Modelos", bbox_to_anchor=(1.02, 1), loc="upper left")
plt.tight_layout()
plt.savefig(f"{OUTPUT_DIR}/comparacao_modelos_physalis.png", dpi=300)
plt.close()

#Solanum melongena
df_melongena = df_validacao[df_validacao["planta"] == "Solanum melongena"]
plt.figure(figsize=(12, 6))
sns.set_theme(style="whitegrid")
sns.lineplot(
    data=df_melongena,
    x="motivo",
    y="contagem",
    hue="modelo",
    style="modelo",
    markers=True,      # Ativa os marcadores nos pontos
    dashes=False,      # Mantém as linhas contínuas
    linewidth=2,       # Espessura da linha
    markersize=10      # Tamanho dos pontos de dispersão
)

# plt.title("Comparação dos Modelos para Solanum melongena", fontsize=14,␣
 ↪fontweight="bold", pad=15)
plt.xlabel("Motivos Conservados", fontsize=12, labelpad=10)
plt.ylabel("Número de Ocorrências (Contagem)", fontsize=12, labelpad=10)
```

```
plt.xticks(rotation=45, ha="right")
plt.legend(title="Modelos", bbox_to_anchor=(1.02, 1), loc="upper left")
plt.tight_layout()
plt.savefig(f"{OUTPUT_DIR}/comparacao_modelos_melongena.png", dpi=300)
plt.close()

#Solanum tuberosum
df_tuberosum = df_validacao[df_validacao["planta"] == "Solanum tuberosum"]
plt.figure(figsize=(12, 6))
sns.set_theme(style="whitegrid")
sns.lineplot(
    data=df_tuberosum,
    x="motivo",
    y="contagem",
    hue="modelo",
    style="modelo",
    markers=True,      # Ativa os marcadores nos pontos
    dashes=False,      # Mantém as linhas contínuas
    linewidth=2,       # Espessura da linha
    markersize=10      # Tamanho dos pontos de dispersão
)

# plt.title("Comparação dos Modelos para Solanum tuberosum", fontsize=14,␣
  ↪fontweight="bold", pad=15)
plt.xlabel("Motivos Conservados", fontsize=12, labelpad=10)
plt.ylabel("Número de Ocorrências (Contagem)", fontsize=12, labelpad=10)
plt.xticks(rotation=45, ha="right")
plt.legend(title="Modelos", bbox_to_anchor=(1.02, 1), loc="upper left")
plt.tight_layout()
plt.savefig(f"{OUTPUT_DIR}/comparacao_modelos_tuberosum.png", dpi=300)
plt.close()

#Arabidopsis thaliana
df_arabidopsis = df_validacao[df_validacao["planta"] == "Arabidopsis thaliana"]
plt.figure(figsize=(12, 6))
sns.set_theme(style="whitegrid")
sns.lineplot(
    data=df_arabidopsis,
    x="motivo",
    y="contagem",
    hue="modelo",
    style="modelo",
    markers=True,      # Ativa os marcadores nos pontos
    dashes=False,      # Mantém as linhas contínuas
    linewidth=2,       # Espessura da linha
    markersize=10      # Tamanho dos pontos de dispersão
)
```

```
# plt.title("Comparação dos Modelos para Arabidopsis thaliana", fontsize=14,␣
 ↪fontweight="bold", pad=15)
plt.xlabel("Motivos Conservados", fontsize=12, labelpad=10)
plt.ylabel("Número de Ocorrências (Contagem)", fontsize=12, labelpad=10)
plt.xticks(rotation=45, ha="right")
plt.legend(title="Modelos", bbox_to_anchor=(1.02, 1), loc="upper left")
plt.tight_layout()
plt.savefig(f"{OUTPUT_DIR}/comparacao_modelos_arabidopsis.png", dpi=300)
plt.close()

#Heatmap
df_validacao["grupo"] = (
    df_validacao["planta"]
    + "_"
    + df_validacao["modelo"]
)

pivot = df_validacao.pivot_table(
    index="motivo",
    columns="grupo",
    values="contagem",
    fill_value=0
)


plt.figure(figsize=(14, 8))
sns.heatmap(
    pivot,
    annot=True,
    fmt="g",
    cmap="YlGnBu",
    linewidths=0.5,
    cbar_kws={'label': 'Contagem'} # Nomeia a barra de cores lateral
)

# plt.title("Distribuição dos Motivos Conservados por Planta e Modelo",␣
 ↪fontsize=14, fontweight="bold", pad=15)
plt.ylabel("Motivo", fontsize=12)
plt.xlabel("Grupo (Planta & Modelo)", fontsize=12)
plt.xticks(rotation=45, ha="right", fontsize=10)
plt.yticks(rotation=0, fontsize=10)
plt.tight_layout()
plt.savefig(f"{OUTPUT_DIR}/heatmap_distribuicao_dos_motivos_conservados.
 ↪png",dpi=300)
plt.close()
# plt.show()
```

```
print("========= Fim do Pipeline ============")
```

```
========= Fim do Pipeline ============
```